

IMPERIAL

IMPERIAL COLLEGE LONDON

DEPARTMENT OF MATHEMATICS

SECOND-YEAR GROUP RESEARCH PROJECT

Langevin Monte Carlo: From Theory to Implementation and Kinetic Extensions

Author:

Jiarui Yan (CID: 02601953)

Paul Wang (CID: 02599058)

Michael Chockler (CID:

02385539)

Max Chong (CID: 02378708)

Cihan Moco (CID: 02597666)

Supervisor(s):

Randolf Altmeyer

September 2, 2026

Abstract

Langevin Monte Carlo provides a principled approach to sampling from Bayesian posterior distributions by exploiting gradient information of the target. However, the standard overdamped algorithm scales unfavourably with dimension and required precision. This project studies three Langevin-based samplers: unadjusted Langevin algorithm (ULA), Metropolis-adjusted Langevin algorithm (MALA), and kinetic Langevin Monte Carlo (KLMC) through theory and implementation. We develop a unified codebase and benchmark on analytically tractable targets (Gaussian and Bayesian linear regression) and a realistic non-Gaussian posterior (Bayesian logistic regression). On these benchmarks, we find that the sampler with the highest mixing rate is not necessarily the most accurate: ULA achieves large effective sample size at large step sizes but suffers discretisation bias, while MALA incurs computational cost through Metropolis corrections but gives reliable posterior moment estimation. We additionally implement and validate the kinetic Langevin integrator of Dalalyan and Riou-Durand [1], testing three theoretical predictions: optimal friction $\gamma^* = \sqrt{m + M}$, dimension scaling as $\sqrt{d}$, and precision scaling as $1/\varepsilon$. The experiments support these predictions qualitatively, though asymptotic constants prevent KLMC from outperforming LMC on the moderate-dimensional, well-conditioned problems tested. The main finding is that rigorous sampler comparison requires balancing accuracy, mixing rate, and computational cost.

Contents

1	Motivation of Langevin MCMC	3
1.1	Why Sampling and Why Langevin Methods?	3
1.2	Why Does Langevin MCMC Work?	5
1.3	Why Study Gaussian Benchmarks, Logistic Regression, and Kinetic Extensions?	6
1.4	Why Langevin in High Dimensions?	8
2	Theoretical Background	8
2.1	The Sampling Problem and Target Distributions	8
2.2	MCMC Fundamentals	9
2.3	From SDEs to Langevin Diffusion	9
2.4	Invariant Distribution of the Langevin Diffusion	9
2.5	Discretisation and the Unadjusted Langevin Algorithm (ULA)	10
2.6	Metropolis Adjustment and MALA	10
2.7	Convergence intuition and key assumptions	10
2.8	Linear Gaussian regression as an explicit benchmark	11
2.9	Bayesian logistic regression	12
3	Implementation	13
3.1	A common sampler interface	14
3.2	The unadjusted Langevin algorithm	14
3.3	The Metropolis-adjusted Langevin algorithm	15
3.4	Mixing diagnostics	16
4	Extension: Kinetic Langevin Monte Carlo	16
4.1	Motivation: From Overdamped to Underdamped Dynamics	16
4.2	The Kinetic Langevin Diffusion	17
4.3	Discretisation: The KLMC Algorithm	18
4.4	Convergence Theory	20
4.5	Validation of Theory	21
4.6	Beyond the Strongly Log-Concave Setting	21
4.7	Discussion	22
5	Results	22
5.1	A first benchmark: a strongly correlated Gaussian	23
5.2	Bayesian linear regression	25
5.3	Bayesian logistic regression	26
5.4	KLMC: Friction, Dimension, and Precision Scaling	28

6	Discussion	31
7	Conclusion	32

1 Motivation of Langevin MCMC

This section explains why sampling is needed in Bayesian computation and why Langevin-based methods provide a natural way to construct efficient samplers. We first recall that posterior quantities are usually expressed as expectations under a target distribution, so direct computation becomes an integration problem. We then explain why generic MCMC methods can be inefficient in high dimensions when they ignore the local geometry of the target. Finally, we motivate Langevin Monte Carlo as a gradient-informed method which combines directional information with stochastic exploration.

Throughout the report, we write $x \in \mathbb{R}^d$ for the unknown parameter or state variable, y for observed data, and π for the target distribution, typically a Bayesian posterior distribution.

Our exposition follows standard references on Langevin sampling, in particular Roberts and Tweedie [2], Dalalyan [3], Durmus and Moulines [4], and Dalalyan and Riou-Durand [1].

1.1 Why Sampling and Why Langevin Methods?

In many problems in modern statistics and machine learning, the aim is not merely to identify a single plausible parameter value (such as a MLE), but to describe an entire probability distribution over an unknown quantity. This viewpoint is particularly natural in Bayesian inference. If $x \in \mathbb{R}^d$ denotes an unknown parameter and y the observed data, then the posterior distribution is given by

$$\pi(x | y) \propto p(y | x)\pi_0(x),$$

where $p(y | x)$ is the likelihood and $\pi_0(x)$ is the prior. In general, the normalising constant

$$p(y) = \int p(y | x)\pi_0(x) dx$$

is analytically intractable, especially in moderate or high dimensions. Nevertheless, most quantities of statistical interest can be expressed as expectations under the posterior distribution. Typical examples include posterior means, posterior variances, marginal probabilities, and posterior predictive quantities. Thus the computational problem is naturally reduced to evaluating, or approximating, integrals of the form

$$\mathbb{E}_\pi[g(X)] = \int g(x)\pi(x) dx.$$

This already suggests why sampling is useful. If one can generate samples approximately distributed according to π , then such expectations may be estimated by Monte Carlo averages,

$$\frac{1}{N} \sum_{i=1}^N g(X_i),$$

which are often far more tractable than direct deterministic integration in high dimension.

A basic non-SDE alternative is importance sampling. Suppose that the target density is known up to a normalising constant, say $\pi(x) \propto \tilde{\pi}(x)$, and that one can sample from a proposal density q . Then expectations under π can be estimated using self-normalised importance weights

$$w_i = \frac{\tilde{\pi}(Z_i)}{q(Z_i)}, \quad Z_i \sim q,$$

through

$$\mathbb{E}_\pi[g(X)] \approx \frac{\sum_{i=1}^N w_i g(Z_i)}{\sum_{i=1}^N w_i}.$$

This method is simple and does not require a Markov chain. However, its performance depends heavily on the choice of the proposal q . In high-dimensional Bayesian problems, constructing a proposal that matches the posterior well can be difficult, and poor proposals may lead to highly variable weights. MCMC methods take a different route: instead of requiring independent samples from a carefully chosen global proposal, they construct a dependent sequence whose long-run distribution is the target. Langevin-based methods further refine this idea by using gradient information to make the Markov chain move in directions informed by the local geometry of the target. The need for sampling is therefore not accidental or merely algorithmic: it is a direct consequence of the fact that Bayesian inference is fundamentally distributional. A point estimate may summarise one aspect of the posterior, but it cannot by itself capture uncertainty, tail behaviour, or dependence between parameters. The reason is that a single value, such as a mode or posterior mean, ignores how probability mass is distributed around that value. For example, two posterior distributions may have the same mode but very different tail behaviour: one may be sharply concentrated around the mode, while another may assign substantial probability to values far away from it. These two posteriors would lead to very different uncertainty quantification, even though they share the same most likely value.

At this stage, one may still ask why a specialised method such as Langevin Monte Carlo is needed at all. Classical MCMC methods such as Metropolis–Hastings are very general, but their practical efficiency depends strongly on the proposal mechanism. In high-dimensional problems, proposals that ignore the geometry of the target can explore the state space very slowly.

This limitation is especially significant when the target density is smooth. In many statistical models, including Bayesian regression models, the target distribution can be written in the form

$$\pi(x) \propto e^{-f(x)},$$

where f is a smooth potential function, typically the negative log-posterior up to an additive constant. Once the target is expressed in this way, gradient information becomes available through

$$\nabla \log \pi(x) = -\nabla f(x).$$

This gradient provides local geometric information about the target distribution: it indicates the direction in which the density increases most rapidly, or equivalently, the direction in which the potential decreases most rapidly. It is therefore natural to ask whether this information can be exploited algorithmically. From the perspective of optimisation, the answer is immediate: one would use gradient descent to move toward a minimiser of f . From the perspective of sampling, however, the challenge is subtler. A deterministic gradient method is designed to converge to a mode, while a sampling method must continue to explore the full distribution rather than collapse onto a single point.

The appeal of Langevin-based methods lies precisely in the way they reconcile these two objectives. They retain the directional information supplied by the gradient, but combine it with random perturbations so that movement is guided rather than purely deterministic. In this sense, Langevin methods sit naturally between optimisation and Monte Carlo. They are more structured than generic random-walk samplers, because they exploit the local shape of the target, yet they remain genuinely stochastic and therefore suited to sampling rather than point estimation. This connection is one of the main conceptual reasons that Langevin Monte Carlo has become such a central object in modern computational statistics.

When the gradient of the log-target can be computed efficiently, it is natural to use it rather than rely on uninformed proposals. In this sense, smoothness of the target distribution is not merely a technical assumption, but a computational resource.

The relevance of this observation becomes even clearer when one compares the objectives of optimisation and sampling more directly. If the task were only to compute a maximiser of the estimate, then one would aim to solve

$$x^* \in \arg \min_{x \in \mathbb{R}^d} f(x),$$

and classical gradient methods would be perfectly natural. But if the aim is instead to approximate expectations under π , then simply descending toward x^* is not enough. One needs a stochastic dynamic that is attracted to regions of high probability without becoming trapped at a single mode. This is the basic idea that underlies Langevin methods: rather than replacing optimisation entirely, they modify it into a stochastic exploration scheme adapted to the target distribution.

This perspective leads naturally to the Langevin diffusion and its time-discretised algorithms, which are introduced next.

The central question, then, is how this local geometric information can be turned into a stochastic process whose long-time behaviour reproduces the target law. The next subsection explains how this idea leads naturally to Langevin dynamics as a stochastic counterpart of gradient-based optimisation.

1.2 Why Does Langevin MCMC Work?

The basic idea behind Langevin Monte Carlo is to turn the local geometric information contained in the gradient of the log-target into a stochastic dynamics whose long-time behaviour reproduces the desired distribution. Under the representation introduced above, the gradient $-\nabla f(x)$ points locally in the direction of increasing density. A deterministic algorithm would use this information to descend toward a minimiser of f , but as discussed above, this would only identify a mode rather than explore the full law. The key step is therefore to combine gradient-driven motion with random perturbations in such a way that the resulting process favours high-probability regions while still retaining global exploration.

This leads naturally to the overdamped Langevin diffusion

$$dX_t = -\nabla f(X_t) dt + \sqrt{2} dW_t,$$

where $(W_t)_{t \geq 0}$ is a standard Brownian motion in $\mathbb{R}^d$. The interpretation of this equation is intuitive. The drift term $-\nabla f(X_t)$ pulls the process toward regions where f is smaller and π is larger, while the Brownian term continuously injects randomness into the dynamics. Thus the process neither wanders completely aimlessly nor converges deterministically to a single minimiser. Instead, it balances attraction toward high-density regions with stochastic exploration. In this sense, Langevin dynamics can be viewed as a noisy gradient flow specifically designed for sampling.

The reason this particular stochastic differential equation is useful is that, under suitable regularity assumptions, the target distribution π is an invariant distribution of the diffusion[2–4]. At an intuitive level, this means that if the process is allowed to evolve for a long time, then its law should settle to the desired target law. Intuitively, this comes from the balance between drift and diffusion: the gradient term pulls mass inward toward regions of high probability, while the Brownian motion spreads mass outward. The corresponding Fokker–Planck verification is given later in Section 2.4.

A correct invariant distribution by itself is not enough for sampling: one also needs convergence toward that distribution from a broad class of initial conditions. The structural assumptions used to quantify this convergence, such as smoothness and strong convexity of the potential, are discussed later in the theoretical background.

The next question is practical rather than conceptual: even if the continuous-time diffusion has the right equilibrium law, how does one actually simulate it? In general, exact simulation is unavailable, so one replaces the diffusion by a time-discretised approximation. The simplest choice is the Euler–Maruyama scheme, which leads to the recursion

$$X_{k+1} = X_k - h\nabla f(X_k) + \sqrt{2h}\xi_{k+1}, \quad \xi_{k+1} \sim N(0, I_d),$$

with step size $h > 0$. This is the unadjusted Langevin algorithm, also known as Langevin Monte Carlo. The structure of the algorithm makes the connection with optimisation very clear: without the Gaussian perturbation it reduces to gradient descent, while with the perturbation it becomes a stochastic exploration scheme. This is the computational form in which Langevin dynamics is usually implemented.

At this point one should be careful to distinguish the idealised diffusion from its discretised approximation. The continuous-time process is designed so that π is invariant, but the discrete-time algorithm produced by Euler discretisation does not in general preserve π exactly. Therefore, Langevin Monte Carlo works only approximately, and its accuracy depends on two effects. The first is a *mixing error*: even the exact diffusion approaches equilibrium only asymptotically, so if it is run for a finite time horizon, its law is not yet exactly equal to π . The second is a *discretisation error*: the iterates (X_k) only approximate the underlying diffusion, and this numerical approximation introduces bias. A principled analysis of Langevin Monte Carlo must therefore control both the time needed for the continuous-time process to approach equilibrium and the extent to which time discretisation perturbs this behaviour.

This decomposition separates the sampling problem into two conceptually distinct pieces: a probabilistic question about convergence to equilibrium and a numerical question about the accuracy of the discretisation. This limitation also motivates the later study of analytically tractable Gaussian benchmarks and kinetic Langevin methods. The next subsection explains why these directions, together with Bayesian logistic regression, are natural choices for this project.

1.3 Why Study Gaussian Benchmarks, Logistic Regression, and Kinetic Extensions?

Once the basic logic of Langevin Monte Carlo is understood, the next question is how to study it in a way that is both mathematically informative and computationally relevant. This project pursues that goal along three complementary directions. First, we consider Gaussian benchmark models, where the target distribution is simple enough to allow explicit calculations. Second, we investigate Bayesian logistic regression, which provides a more realistic and practically important example of a smooth posterior distribution. Third, we examine kinetic Langevin Monte Carlo as a natural extension of the overdamped framework, motivated by the limitations of the unadjusted Langevin algorithm in high dimension and at high accuracy.

Gaussian targets play a particularly important role because they provide an analytically tractable setting in which the behaviour of Langevin Monte Carlo can be understood in detail. When the target distribution is Gaussian, the gradient of the potential is linear, the corresponding Langevin diffusion is an Ornstein–Uhlenbeck-type process, and many quantities of interest can be computed explicitly. This makes Gaussian models an ideal benchmark for at least two reasons. On the theoretical side, they allow one to prove convergence statements directly rather

than only through abstract inequalities. In particular, a linear Gaussian regression model leads to a Gaussian posterior, and this makes it possible to analyse the Langevin iteration with a level of precision that would be difficult to achieve in a fully nonlinear setting. On the computational side, Gaussian models provide a clean testing ground in which empirical results can be compared against exact posterior means and covariances, thereby making it easier to diagnose whether an implementation is behaving as expected.

However, Gaussian models alone are not sufficient if one wants to understand how Langevin Monte Carlo behaves in problems that are closer to contemporary statistical applications. For this reason, Bayesian logistic regression is also a natural object of study. Logistic regression is a standard and widely used model for binary classification. In the Bayesian setting, it gives a posterior distribution that is typically non-Gaussian, yet still smooth enough for gradient-based methods to be meaningful and effective. In this model, the posterior combines the logistic likelihood with a prior distribution, yielding a smooth potential of the same form as introduced above. Under suitable regularisation, this potential is strongly convex, so Bayesian logistic regression remains connected to the theoretical framework developed later in the report. This makes Bayesian logistic regression especially useful as an intermediate case between fully explicit Gaussian benchmarks and more complicated modern models. It is rich enough to illustrate the practical strengths and weaknesses of Langevin Monte Carlo, yet structured enough to connect naturally with the theoretical framework developed in the literature.

The contrast between the Gaussian and logistic settings also helps clarify what different parts of the project are meant to achieve. In the Gaussian case, one can hope for explicit proofs and exact comparisons, since the target law is completely tractable. In the logistic case, by contrast, the emphasis shifts toward implementation and empirical behaviour: one studies how the algorithm performs numerically, how sensitive it is to the choice of step size, and how well theoretical predictions reflect actual sampling performance. Together, these two examples provide a useful balance between theory and practice. One shows the mechanism of the method in its clearest algebraic form; the other shows how the same ideas operate in a more realistic Bayesian computation problem.

A further motivation for this project comes from the limitations of the overdamped Langevin algorithm. Because Brownian noise acts directly on the position variable, the resulting paths are rough, and controlling discretisation error may require small step sizes. This motivates kinetic, or underdamped, Langevin dynamics, where an auxiliary velocity variable is introduced and the noise acts through the velocity. This preserves the same basic philosophy of using gradient information for stochastic exploration, while providing a natural direction for improving the underlying dynamics.

For the present project, this extension is especially interesting because it connects naturally with the earlier analysis of overdamped Langevin Monte Carlo. Once the reader understands why the overdamped method works, it becomes meaningful to ask whether its convergence behaviour can be improved by modifying the underlying dynamics. The results of Dalalyan and Riou-Durand suggest that this is indeed the case: kinetic Langevin methods can achieve better dependence on the dimension and on the target precision, at least in the strongly log-concave setting [1]. Thus the extension is not merely an add-on, but a natural continuation of the main question of the project.

In summary, the choice of Gaussian benchmarks, logistic regression, and kinetic Langevin extensions is guided by a common principle. We would like to understand Langevin Monte Carlo at three levels: in a setting where it can be analysed exactly, in a setting where it can be tested in a realistic Bayesian computation problem, and in a setting where its underlying dynamics can be improved. This is why the project is organised as it is. We first motivate and explain the overdamped Langevin method, then study its behaviour analytically and numerically through

Gaussian and logistic examples, and finally turn to kinetic Langevin Monte Carlo as a principled extension of the same sampling philosophy.

1.4 Why Langevin in High Dimensions?

A natural question is why introduce Langevin dynamics at all, since approximate sampling can already be performed by MCMC methods such as the Metropolis–Hastings. The key point is that Metropolis–Hastings is only a framework: its practical performance depends on how the proposal distribution is chosen. In particular, if one uses a proposal that does not exploit the local geometry of the target distribution, then the resulting chain may explore the state space very inefficiently.

This issue becomes especially important in high-dimensional problems. For a high-dimensional uninformed random walk, our proposal typically needs to take small steps in order to maintain a reasonable acceptance probability. As a result, the chain may require many iterations to move across the region of the target distribution. In contrast, Langevin-based methods incorporate the gradient of the log-density and therefore use local information about the shape of the target. The gradient term guides the proposal toward regions of higher probability, while the Gaussian perturbation maintains stochastic exploration.

Therefore, Langevin methods are not replacing the Metropolis idea, but rather providing a more informed proposal mechanism. Indeed, the unadjusted Langevin algorithm may be viewed as a direct discretisation of the Langevin diffusion, while the Metropolis-adjusted Langevin algorithm combines the same gradient-based proposal with an accept–reject correction. The main attraction of Langevin methods is therefore not that they are the only possible MCMC approach, but that they are particularly well suited to smooth high-dimensional targets, where geometric information can substantially improve efficiency.

2 Theoretical Background

This section introduces the main theoretical framework used throughout the report. We begin by formulating the sampling problem and introducing the basic MCMC viewpoint. We then focus on Langevin-based sampling methods, starting from the continuous-time Langevin diffusion and its invariant distribution, and then passing to its time discretisation, which gives the unadjusted Langevin algorithm. We also briefly discuss the Metropolis-adjusted version and summarise the assumptions that commonly appear in convergence analysis. Finally, we explain why linear Gaussian regression and Bayesian logistic regression provide natural benchmark models for the later theoretical and empirical study.

2.1 The Sampling Problem and Target Distributions

We consider the problem of sampling from a probability distribution π on $\mathbb{R}^d$. In many statistical applications, especially in Bayesian inference, the target distribution is a posterior law of the form

$$\pi(x \mid y) \propto p(y \mid x)p(x),$$

where $p(y \mid x)$ is the likelihood and $p(x)$ is the prior. Since the normalising constant is often intractable, the density is typically known only up to proportionality. For Langevin-based methods, it is convenient to write the target distribution in the form

$$\pi(x) \propto e^{-f(x)},$$

where $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is the potential function. In the Bayesian setting, f is the negative log-posterior up to an additive constant. This representation is particularly useful because the gradient ∇f determines the drift in the Langevin dynamics. The goal is to construct an algorithm that produces samples whose distribution is close to π , so that expectations of interest can be approximated from the simulated output.

2.2 MCMC Fundamentals

Markov chain Monte Carlo (MCMC) methods construct a Markov chain whose invariant distribution is the target distribution π . If the chain is simulated for sufficiently many steps, its iterates can be used to approximate expectations under π . A common sufficient condition for π to be invariant is the detailed balance condition:

$$\pi(x)P(x, dy) = \pi(y)P(y, dx),$$

where P denotes the transition kernel of the chain. Classical algorithms such as Metropolis–Hastings are designed to satisfy this condition. In the present project, our main interest is not in general MCMC constructions, but in gradient-based methods obtained from Langevin dynamics. These methods replace generic proposal mechanisms by updates that make direct use of the geometry of the target distribution through the gradient of the potential.

2.3 From SDEs to Langevin Diffusion

A general stochastic differential equation on $\mathbb{R}^d$ has the form

$$dX_t = b(X_t) dt + \sigma dW_t,$$

where b is the drift, σ is the diffusion coefficient, and $(W_t)_{t \geq 0}$ is a standard Brownian motion. In the potential representation introduced above, the drift is chosen using the gradient of f . This leads to the overdamped Langevin diffusion

$$dX_t = -\nabla f(X_t) dt + \sqrt{2} dW_t.$$

The drift term drives the process toward regions where f is smaller, while the Brownian term maintains stochastic exploration. Under suitable regularity assumptions, this diffusion has π as its invariant distribution and forms the continuous-time foundation of Langevin Monte Carlo.

2.4 Invariant Distribution of the Langevin Diffusion

Let $\rho_t(x)$ denote the time-marginal density of the diffusion above. A standard calculation for the Langevin diffusion (see, for example, [2]) shows that ρ_t satisfies

$$\partial_t \rho_t(x) = \nabla \cdot (\rho_t(x) \nabla f(x)) + \Delta \rho_t(x).$$

Differentiating the exponential proportionality identity gives

$$\nabla \pi(x) = -\pi(x) \nabla f(x) \implies \nabla \cdot (\pi(x) \nabla f(x)) + \Delta \pi(x) = 0$$

(after a second differentiation). Hence, if $\rho_t = \pi$, then the right-hand side of the Fokker–Planck equation vanishes, so

$$\partial_t \rho_t = 0.$$

This means that π is unchanged by the Fokker–Planck evolution and is therefore a stationary solution. Equivalently, the Langevin diffusion admits π as an invariant distribution [2]. This is the basic mechanism underlying Langevin-based sampling.

2.5 Discretisation and the Unadjusted Langevin Algorithm (ULA)

To obtain a practical algorithm, the Langevin diffusion is discretised in time. With step size $h > 0$, the Euler–Maruyama scheme gives

$$X_{k+1} = X_k - h\nabla f(X_k) + \sqrt{2h}\xi_{k+1}, \quad \xi_{k+1} \sim N(0, I_d).$$

This defines the unadjusted Langevin algorithm (ULA), also known as Langevin Monte Carlo. The deterministic term $-h\nabla f(X_k)$ moves the iterate toward regions of higher probability, while the Gaussian perturbation maintains exploration. Unlike the continuous-time diffusion, however, the discretised chain does not in general preserve the target distribution exactly. Its accuracy therefore depends on both convergence to equilibrium and discretisation error.

2.6 Metropolis Adjustment and MALA

A standard way to remove the discretisation bias of ULA is to add a Metropolis–Hastings correction. Starting from the current state $X_k = x$, one first generates a Langevin proposal and then accepts or rejects it according to a Metropolis rule. The full procedure is summarised in Algorithm 3.

Algorithm 1: Metropolis-adjusted Langevin algorithm (MALA)

Input: current state $X_0 = x_0$; step size h ; iterations N

Output: chain $(X_k)_{k=1}^N$

$X \leftarrow x_0$

for $k = 1$ **to** N **do**

 draw $\xi \sim N(0, I_d)$

$Y \leftarrow X + h\nabla \log \pi(X) + \sqrt{2h}\xi$

 compute $\alpha(X, Y) = \min \left\{ 1, \frac{\pi(Y)q_h(Y, X)}{\pi(X)q_h(X, Y)} \right\}$

 draw $U \sim \text{Uniform}(0, 1)$

if $U \leq \alpha(X, Y)$ **then**

$X \leftarrow Y$

$X_k \leftarrow X$

Here $q_h(x, \cdot)$ denotes the Gaussian proposal density induced by the Langevin step. This yields the Metropolis-adjusted Langevin algorithm (MALA). In contrast to ULA, MALA has the target distribution π exactly as its invariant distribution. In the present project, however, the main focus remains on the unadjusted algorithm, since it is simpler to analyse explicitly in the Gaussian benchmark and provides the natural starting point for comparison with kinetic Langevin methods.

2.7 Convergence intuition and key assumptions

To understand the behaviour of Langevin-based samplers, it is useful to identify the main assumptions under which rigorous convergence guarantees are available.

Throughout the theoretical discussion, we work with the potential representation introduced earlier, wherein the potential function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is assumed to be m -strongly convex and to have an M -Lipschitz continuous gradient. Equivalently,

$$f(x) - f(y) - \nabla f(y)^T(x - y) \geq \frac{m}{2} \|x - y\|^2, \quad (1)$$

$$\|\nabla f(x) - \nabla f(y)\| \leq M \|x - y\|, \quad (2)$$

for all $x, y \in \mathbb{R}^d$ [3]. These imply that the target distribution is strongly log-concave and that the geometry of the potential is sufficiently regular for quantitative convergence analysis.

The ratio $\kappa = M/m$ is known as the condition number of the sampling problem. It measures the balance between the curvature of the target distribution and the smoothness of its gradient.

Theorem 2.1 (Exponential convergence of the Langevin diffusion). *Let $\pi(x) \propto e^{-f(x)}$, where f satisfies (1) and (2). Then the Langevin diffusion*

$$dL_t = -\nabla f(L_t) dt + \sqrt{2} dW_t \quad (3)$$

converges at an exponential rate to its invariant distribution π . In particular, the convergence rate is governed by the strong convexity parameter m [1, 3].

This theorem explains why the parameter m plays a central role in the analysis of Langevin algorithms. The continuous-time diffusion mixes at an exponential rate determined by m , whereas the discretisation error introduced by the Euler scheme underlying ULA depends on the smoothness parameter M and the step size h .

Consequently,

$$\text{total error} \approx \text{mixing error} + \text{discretisation error}. \quad (4)$$

The condition number κ therefore appears naturally in many complexity bounds for Langevin Monte Carlo algorithms [1, 3, 4].

These ideas become particularly transparent in the Gaussian setting, where the potential function is quadratic and all quantities can be computed explicitly. For this reason, Bayesian linear Gaussian regression serves as a useful benchmark before moving to more realistic models such as Bayesian logistic regression.

2.8 Linear Gaussian regression as an explicit benchmark

Linear Gaussian regression is the one nontrivial model in which every quantity appearing in the convergence analysis can be computed in closed form. It therefore serves as a benchmark: the posterior, its potential, the constants m and M , and the exact stationary bias of ULA are all available explicitly, so the predictions of the preceding sections can be verified directly.

Consider the Bayesian linear model with a conjugate Gaussian prior,

$$y = X\beta + \varepsilon, \quad \varepsilon \sim N(0, \sigma^2 I_n), \quad \beta \sim N(m_0, \Sigma_0). \quad (5)$$

By Bayes' theorem $p(\beta | y) \propto p(y | \beta) p(\beta)$, so the negative log-posterior is, up to an additive constant C independent of β ,

$$-\log p(\beta | y) = \frac{1}{2\sigma^2} (y - X\beta)^T (y - X\beta) + \frac{1}{2} (\beta - m_0)^T \Sigma_0^{-1} (\beta - m_0) + C.$$

This is quadratic in β , so the posterior is again Gaussian. Completing the square gives $\beta | y \sim N(m_n, \Sigma_n)$, with

$$\Sigma_n^{-1} = \Sigma_0^{-1} + \frac{1}{\sigma^2} X^T X, \quad m_n = \Sigma_n \left(\Sigma_0^{-1} m_0 + \frac{1}{\sigma^2} X^T y \right). \quad (6)$$

Writing the posterior in the form $\pi(\beta) \propto e^{-f(\beta)}$, the potential and its derivatives are

$$f(\beta) = \frac{1}{2}(\beta - m_n)^T \Sigma_n^{-1}(\beta - m_n), \quad \nabla f(\beta) = \Sigma_n^{-1}(\beta - m_n), \quad \nabla^2 f(\beta) = \Sigma_n^{-1}, \quad (7)$$

where the additive constant has been absorbed into the normalisation. Because the Hessian is the constant matrix Σ_n^{-1} , the strong-convexity and smoothness constants are exactly its extreme eigenvalues, and the condition number of the sampling problem is that of the posterior precision matrix:

$$m = \lambda_{\min}(\Sigma_n^{-1}), \quad M = \lambda_{\max}(\Sigma_n^{-1}), \quad \kappa = \frac{M}{m} = \frac{\lambda_{\max}(\Sigma_n^{-1})}{\lambda_{\min}(\Sigma_n^{-1})}.$$

Applying the ULA update (14) to the quadratic potential (7) yields a recursion that is affine in β :

$$\beta_{k+1} = \beta_k - h \Sigma_n^{-1}(\beta_k - m_n) + \sqrt{2h} \xi_{k+1} = A\beta_k + b + \sqrt{2h} \xi_{k+1}, \quad \xi_{k+1} \sim N(0, I_d), \quad (8)$$

with $A = I_d - h\Sigma_n^{-1}$ and $b = h\Sigma_n^{-1}m_n$. This linearity is what makes the Gaussian case tractable: the mean and covariance of β_k each satisfy a closed recursion.

Since $Am_n + b = m_n$, the posterior mean is the fixed point of the affine map, and subtracting it gives $\mathbb{E}[\beta_{k+1}] - m_n = A(\mathbb{E}[\beta_k] - m_n)$. The eigenvalues of A are $1 - h\lambda_i$, where the λ_i are the eigenvalues of Σ_n^{-1} , so $\mathbb{E}[\beta_k] \rightarrow m_n$ geometrically at rate $\rho(A) = \max_i |1 - h\lambda_i|$ precisely when $0 < h < 2/\lambda_{\max}(\Sigma_n^{-1})$. The role of the condition number is now explicit: when κ is large the eigenvalues of Σ_n^{-1} are widely spread, so no single step size is simultaneously large and stable in every direction, and the chain mixes slowly.

Because ξ_{k+1} is independent of β_k , the covariance $C_k = \text{Cov}(\beta_k)$ obeys

$$C_{k+1} = A C_k A^T + 2h I_d. \quad (9)$$

Whenever $\rho(A) < 1$ this has a unique stationary solution C_* , characterised by the discrete Lyapunov equation $C_* = A C_* A^T + 2h I_d$. For any finite step size $C_* \neq \Sigma_n$: the unadjusted chain converges to a Gaussian with the correct mean but a distorted covariance. This is exactly the finite-step bias of ULA, and it vanishes as $h \rightarrow 0$, where $C_* \rightarrow \Sigma_n$.

In summary, linear Gaussian regression separates the two error sources cleanly. The mixing rate is governed by the spectrum of Σ_n^{-1} , hence by κ , via (8); the discretisation bias is the gap $C_* - \Sigma_n$ implied by (9), which shrinks to zero with the step size. Both are computable in closed form, which is what makes this model the natural benchmark before the non-Gaussian logistic case.

2.9 Bayesian logistic regression

Bayesian logistic regression is one of the most widely used models for binary classification and therefore provides a practically important non-Gaussian benchmark for Langevin-based sampling algorithms.

Unlike the linear Gaussian model considered in the previous section, the posterior distribution is no longer available in closed form. Although the log-posterior and its gradient can still be evaluated efficiently, the potential function is non-quadratic and its exact moments are generally unavailable. Consequently, the explicit analysis carried out for the Gaussian benchmark is no longer possible.

Nevertheless, the posterior remains smooth and, under mild conditions on the design matrix and prior, is strongly log-concave. Therefore the general convergence theory discussed in Section 2.7 still applies, including non-asymptotic convergence guarantees for ULA, MALA and related Langevin algorithms [3, 4].

Given covariates $x_i \in \mathbb{R}^d$ and binary observations $y_i \in \{0, 1\}$, the model is

$$y_i \mid x_i, \beta \sim \text{Bernoulli}(\sigma(x_i^T \beta)), \quad (10)$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the logistic sigmoid function.

With a Gaussian prior $\beta \sim N(0, \tau^2 I)$, the posterior density is proportional to

$$\pi(\beta \mid X, y) \propto \prod_{i=1}^n \sigma(x_i^T \beta)^{y_i} (1 - \sigma(x_i^T \beta))^{1-y_i} \exp\left(-\frac{\|\beta\|^2}{2\tau^2}\right). \quad (11)$$

Taking logarithms gives the negative log-posterior

$$f(\beta) = -\sum_{i=1}^n [y_i \log \sigma(x_i^T \beta) + (1 - y_i) \log(1 - \sigma(x_i^T \beta))] + \frac{\|\beta\|^2}{2\tau^2} \quad (12)$$

which can be differentiated to give

$$\nabla f(\beta) = X^T(\sigma(X\beta) - y) + \frac{1}{\tau^2}\beta. \quad (13)$$

Unlike the Gaussian benchmark, the posterior distribution is not Gaussian and no closed-form expression is available for its moments. Consequently, quantities such as the invariant distribution of the ULA chain, the stationary covariance, and the exact convergence rate cannot be computed analytically.

However, the gradient remains available in closed form, making the posterior well suited to Langevin-based sampling methods. Therefore, in the numerical experiments that follow, we use Bayesian logistic regression as a realistic test problem and compare ULA, MALA and KLMC on a non-Gaussian target distribution.

3 Implementation

The preceding sections motivated the overdamped Langevin diffusion and its Euler–Maruyama discretisation, and showed in the Gaussian case that the unadjusted algorithm converges to a law whose mean is correct but whose covariance carries an h -dependent bias. The purpose of this section is to turn that theory into code, and then to use the code to observe the predicted behaviour directly.

We proceed in two stages. The present half of the section fixes a common interface for the samplers, implements the unadjusted Langevin algorithm (ULA) and its Metropolis-adjusted counterpart (MALA), develops the mixing diagnostics that the later comparisons rely on, and validates everything on an analytically tractable Gaussian benchmark. The second half then applies these tools to Bayesian linear and logistic regression, introduces a cost-aware efficiency metric based on gradient evaluations, and studies the dependence on the step size and the dimension; it also provides the empirical comparison with the kinetic sampler of Section 4.

All code is written in Python using NumPy for the linear algebra and SciPy for the optimisation and special functions. Experiments are driven by an explicit `numpy.random.Generator`, seeded at the point of use, so that all figures in this report are exactly reproducible and so that competing samplers can be compared using matched random seeds where appropriate [5].

3.1 A common sampler interface

Both Langevin samplers require only two pieces of information about the target: the value of the (unnormalised) log-density and its gradient. We therefore represent a target by a pair of callables,

$$\text{log_prob} : x \mapsto \log \pi(x) + \text{const}, \quad \text{grad_log_prob} : x \mapsto \nabla \log \pi(x),$$

both acting on a point $x \in \mathbb{R}^d$. Working with $\log \pi$ and $\nabla \log \pi$ rather than with the potential f is a purely notational convenience that matches the way targets are specified in practice: since $\pi(x) \propto e^{-f(x)}$ we have $\nabla \log \pi = -\nabla f$, and the unadjusted recursion of Section 1,

$$X_{k+1} = X_k - h \nabla f(X_k) + \sqrt{2h} \xi_{k+1}, \quad \xi_{k+1} \sim N(0, I_d),$$

becomes, in these terms,

$$X_{k+1} = X_k + h \nabla \log \pi(X_k) + \sqrt{2h} \xi_{k+1}. \quad (14)$$

Two points about the interface are worth stating, because they recur throughout the experiments. First, only the unnormalised log-density is ever needed: every algorithm below depends on π through differences $\log \pi(y) - \log \pi(x)$ or through the gradient, so the intractable normalising constant cancels and never has to be computed. Second, because the gradient is the only expensive operation in realistic models, the number of gradient evaluations is the natural unit of computational cost; we return to this point when comparing samplers in Section 5.

3.2 The unadjusted Langevin algorithm

The unadjusted Langevin algorithm is a direct transcription of the Euler–Maruyama recursion (14). Given a step size $h > 0$, a starting point x_0 , and a number of iterations N , it repeatedly takes a gradient step and adds Gaussian noise of variance $2h$ in each coordinate, recording the position after every step. The procedure is summarised in Algorithm 2.

Algorithm 2: Unadjusted Langevin algorithm (ULA)

Input: gradient $\nabla \log \pi$; start x_0 ; step size h ; iterations N

Output: samples $(X_k)_{k=1}^N$

$X \leftarrow x_0$

for $k = 1$ **to** N **do**

 draw $\xi \sim N(0, I_d)$

$X \leftarrow X + h \nabla \log \pi(X) + \sqrt{2h} \xi$

$X_k \leftarrow X$

The implementation is deliberately minimal: it performs exactly one gradient evaluation per iteration, allocates the noise scale $\sqrt{2h}$ once outside the loop, and pre-allocates the output array. Its simplicity is exactly the point. ULA carries no accept/reject step, so it never stalls, but it also never corrects the discretisation error introduced by the Euler scheme. As established

in Section 1 and made precise in the Gaussian case below, the chain produced by Algorithm 2 converges to a stationary law that differs from π by a bias of order h [3, 4]. The practical consequence is that ULA is accurate only when h is small, and small h in turn slows exploration; the experiments below quantify this trade-off.

3.3 The Metropolis-adjusted Langevin algorithm

To remove the discretisation bias we treat the Langevin step as a *proposal* and accept or reject it with a Metropolis–Hastings correction [2]; we follow the construction as presented in the lecture notes of Duncan [6]. From the current state x the proposal

$$y = x + h \nabla \log \pi(x) + \sqrt{2h} \xi, \quad \xi \sim N(0, I_d), \quad (15)$$

is drawn from a Gaussian whose mean $\mu(x) = x + h \nabla \log \pi(x)$ depends on the current gradient and whose covariance is $2h I_d$. The corresponding proposal density is

$$q_h(x, y) \propto \exp\left(-\frac{1}{4h} \|y - \mu(x)\|^2\right),$$

and the proposal is accepted with probability

$$\alpha(x, y) = \min\left\{1, \frac{\pi(y) q_h(y, x)}{\pi(x) q_h(x, y)}\right\}. \quad (16)$$

Two implementation details make this efficient and numerically robust, and both are visible in Algorithm 3. First, the acceptance ratio is evaluated on the logarithmic scale, so that $\log \alpha = (\log \pi(y) - \log \pi(x)) + (\log q_h(y, x) - \log q_h(x, y))$, with the proposal increment computed as $\log q_h(x, y) = -\|y - \mu(x)\|^2 / (4h)$ up to a constant $1/(4\pi h)^{d/2}$ that is identical in the forward and reverse densities and therefore cancels. Working in logs avoids overflow in the density evaluations and lets the accept/reject test be written as $\log U < \log \alpha$ with $U \sim \text{Uniform}(0, 1)$. Second, the gradient at an accepted state is cached and reused as the current gradient on the next iteration, since the reverse proposal mean $\mu(y) = y + h \nabla \log \pi(y)$ must be formed from the gradient at the proposed point in any case.

Algorithm 3: Metropolis-adjusted Langevin algorithm (MALA)

Input: log-target $\log \pi$ and gradient $\nabla \log \pi$; start x_0 ; step size h ; iterations N

Output: samples $(X_k)_{k=1}^N$; acceptance rate

$x \leftarrow x_0$; $\ell_x \leftarrow \log \pi(x)$; $g_x \leftarrow \nabla \log \pi(x)$; $a \leftarrow 0$

for $k = 1$ **to** N **do**

$\mu_x \leftarrow x + h g_x$; draw $\xi \sim N(0, I_d)$; $y \leftarrow \mu_x + \sqrt{2h} \xi$

$\ell_y \leftarrow \log \pi(y)$; $g_y \leftarrow \nabla \log \pi(y)$; $\mu_y \leftarrow y + h g_y$

$\log \alpha \leftarrow (\ell_y - \ell_x) - \frac{1}{4h} (\|x - \mu_y\|^2 - \|y - \mu_x\|^2)$

 draw $U \sim \text{Uniform}(0, 1)$

if $\log U < \log \alpha$ **then**

$x \leftarrow y$; $\ell_x \leftarrow \ell_y$; $g_x \leftarrow g_y$; $a \leftarrow a + 1$

$X_k \leftarrow x$

The price of the correction is one additional gradient evaluation per iteration: the proposed point's gradient is required both for the reverse proposal density in (16) and as the cached gradient should the move be accepted. MALA therefore costs roughly twice as much per step as ULA, a difference we account for explicitly when we measure efficiency per gradient evaluation later. In exchange, MALA has π as its *exact* invariant distribution, so its only error is the Monte

Carlo variance of a finite run rather than a systematic bias. The tension between ULA's cheaper, biased steps and MALA's corrected but occasionally rejected steps is the central theme of the comparisons that follow.

3.4 Mixing diagnostics

Comparing samplers requires more than checking that their sample moments are close to the truth: a chain can be unbiased and yet explore the target so slowly that a finite run is almost useless, or it can move quickly while sampling the wrong law. We therefore measure two distinct things, accuracy and mixing, and keep them separate throughout. Accuracy, in the cases where the truth is known, is measured by the distance of the sample moments from their exact values. For a target with mean μ and covariance Σ we report the mean error $\|\hat{\mu} - \mu\|_2$ and the covariance error $\|\hat{\Sigma} - \Sigma\|_F$ in the Frobenius norm, computed from the post-burn-in samples. Mixing is summarised by the *effective sample size* (ESS), which estimates how many independent draws a correlated chain of length N is worth. For a one-dimensional chain (x_k) with normalised autocorrelation function ρ_k , the integrated autocorrelation time

$$\tau = 1 + 2 \sum_{k \geq 1} \rho_k$$

inflates the variance of a sample average relative to the independent case, and the effective sample size is $\text{ESS} = N/\tau$. We estimate τ using Geyer's initial positive sequence rule, summing the autocorrelations in consecutive pairs and truncating as soon as a pair sum becomes non-positive; this guards against the noisy, small-magnitude autocorrelations at large lags, which would otherwise accumulate spurious variance into the estimate. For a multivariate chain we report the *minimum* ESS across coordinates, a deliberately conservative choice that reflects the worst-mixing direction. A small but worthwhile implementation choice concerns the autocorrelation itself. A direct evaluation of ρ_k at every lag costs $O(N^2)$ per coordinate, which is wasteful once τ requires the full autocorrelation curve on chains of tens of thousands of samples. We instead compute the autocorrelation through the fast Fourier transform: zero-padding the centred chain to avoid circular wrap-around, transforming, taking the squared modulus, and inverting recovers all lags at once in $O(N \log N)$. The two routes agree to numerical precision, but the FFT version is what makes the full-lag ESS computation cheap enough to run inside every step-size sweep below.

4 Extension: Kinetic Langevin Monte Carlo

4.1 Motivation: From Overdamped to Underdamped Dynamics

The Langevin Monte Carlo algorithm studied in the preceding sections provides a principled approach to sampling from a target density $\pi(x) \propto e^{-f(x)}$ on $\mathbb{R}^d$. Under the assumption that f is m -strongly convex with M -Lipschitz gradient, the analysis of Dalalyan [3] establishes that, to bring the law ν_K of the K -th iterate within total variation distance ε of π , it is sufficient to choose

$$K \gtrsim \frac{d}{\varepsilon^2} \log\left(\frac{1}{\varepsilon}\right),$$

up to a polynomial factor in the condition number $\kappa = M/m$. This bound, polynomial both in the dimension and in the inverse precision, serves as the baseline for non-asymptotic guarantees in this setting.

Two features of this rate motivate an extension. First, the dependence on the precision ε is quadratic, which becomes unfavorable whenever high-accuracy posterior summaries are required. Second, the dependence on the dimension is linear, which becomes costly in high dimensions even when ε is held fixed. Both limitations stem from the same structural property of the underlying continuous process.

In the overdamped diffusion

$$dL_t = -\nabla f(L_t) dt + \sqrt{2} dW_t,$$

the Brownian noise enters the position variable directly, making the sample paths of L_t irregular at small time scales. The Euler–Maruyama discretisation must therefore use a small step size h to control the local truncation error, and reducing h increases the number of iterations required to traverse a given time horizon.

To circumvent this limitation, we consider a continuous-time process whose sample paths are smoother, while preserving π as a marginal of its invariant law. The kinetic, or underdamped, Langevin diffusion is the canonical process for this purpose. It is defined on the extended state space $\mathbb{R}^d \times \mathbb{R}^d$, with coordinates representing position and velocity, and is governed by the second-order stochastic differential equation

$$d \begin{pmatrix} V_t \\ L_t \end{pmatrix} = \begin{pmatrix} -\gamma V_t - \nabla f(L_t) \\ V_t \end{pmatrix} dt + \sqrt{2\gamma} \begin{pmatrix} I_d \\ 0 \end{pmatrix} dW_t,$$

where $\gamma > 0$ is a friction coefficient. The drift acts directly on the velocity rather than the position. Because noise enters only the velocity component, the position is recovered by integrating the velocity in time. Since integration is a smoothing operation, the position paths of the kinetic process are considerably more regular than those of the overdamped process. Consequently, the discretisation can tolerate a larger step size without a corresponding loss of accuracy [1].

This smoother behaviour translates into an algorithmic advantage. The analysis of Dalalyan and Riou-Durand [1], building on the non-asymptotic framework of Cheng et al. [7], shows that a (suitably) discretised version of the kinetic diffusion attains the error bound $\mathcal{W}_2(\nu_K, \pi) \leq \varepsilon$ after

$$K \gtrsim \frac{\sqrt{d}}{\varepsilon} \kappa^{3/2} \log\left(\frac{1}{\varepsilon}\right)$$

iterations. This rate improves upon the overdamped case by reducing the dimension dependence from d to $\sqrt{d}$ and the precision dependence from ε^{-2} to ε^{-1} .

An analogy can be drawn with gradient-based optimisation, where adding a momentum term to gradient descent accelerates convergence on well-conditioned problems. The kinetic Langevin diffusion operates similarly for sampling: the velocity variable V_t acts as a momentum term, and the friction γ controls the damping of that momentum. As $\gamma \rightarrow \infty$, the dynamics revert to the overdamped case, while a small γ induces undamped oscillation. The choice $\gamma \asymp \sqrt{m + \bar{M}}$ balances these regimes.

4.2 The Kinetic Langevin Diffusion

To address this path-irregularity issue, we can look at a continuous-time process where the paths are smoother. The kinetic Langevin diffusion is a stochastic process on $\mathbb{R}^d \times \mathbb{R}^d$ with state (L_t, V_t) representing position and velocity. It is defined as the strong solution of the stochastic differential equation

$$d \begin{pmatrix} V_t \\ L_t \end{pmatrix} = \begin{pmatrix} -\gamma V_t - \nabla f(L_t) \\ V_t \end{pmatrix} dt + \sqrt{2\gamma} \begin{pmatrix} I_d \\ 0 \end{pmatrix} dW_t, \quad (17)$$

where $\gamma > 0$ is the friction parameter and W_t is a standard d -dimensional Brownian motion. The structure of this SDE differs from the overdamped case in two ways. First, the gradient $\nabla f(L_t)$ acts on the velocity rather than directly on the position, and the position itself evolves via the relation $dL_t = V_t dt$. Second, the Brownian noise enters only the velocity component; there is no direct stochastic forcing of the position. These features produce the smoother sample paths used by the KLMC discretisation in Section 4.3.

Invariant distribution. The diffusion preserves the joint density

$$p_*(x, v) \propto \exp\left(-f(x) - \frac{1}{2}\|v\|_2^2\right), \quad x, v \in \mathbb{R}^d. \quad (18)$$

Because the exponent is a sum of separable terms, the joint density factors as a product:

$$p_*(x, v) \propto e^{-f(x)} \cdot e^{-\frac{1}{2}\|v\|_2^2}.$$

This factorization implies that position and velocity are independent under the invariant distribution. Furthermore, the position marginal, obtained by integrating out v , is exactly the target $\pi \propto e^{-f}$, while the velocity marginal is the standard Gaussian $\mathcal{N}(0, I_d)$. Sampling from π therefore proceeds by simulating the joint SDE (17), recording the position iterates, and discarding the velocity component, which acts as an auxiliary variable.

This invariance can be verified by checking that p_* is a steady-state solution of the Fokker–Planck equation associated with (17), matching the verification outlined for the overdamped case in Section 1.

Role of the friction parameter. The behaviour of the diffusion depends on the magnitude of γ relative to the curvature of f :

- Underdamped configurations (small γ) exhibit weak damping, causing the process to oscillate. In the limit $\gamma \rightarrow 0$, the system becomes a stochastically forced Hamiltonian system which does not mix.
- Conversely, when γ is large, friction dominates the dynamics. Under the time rescaling $\bar{L}_t = L_{\gamma t}$, the trajectories contract to the overdamped Langevin diffusion [8, Theorem 10.1].
- The optimal continuous-time mixing rate occurs between these extremes, where the convergence theory in Section 4.4 balances these regimes at $\gamma \asymp \sqrt{m + \bar{M}}$.

The formulation of Dalalyan and Riou-Durand [1] includes an additional mass parameter $u > 0$. As shown in their Lemma 1, u can be absorbed by a time rescaling of the process, so we set $u = 1$ throughout.

4.3 Discretisation: The KLMC Algorithm

The kinetic Langevin SDE (17) is a continuous-time object and cannot be simulated directly. To implement an algorithm, we require an update rule that takes the state at time kh and returns the state at time $(k+1)h$. Rather than using a standard Euler–Maruyama scheme, we follow the discretisation of Dalalyan and Riou-Durand [1], which exploits the structure of the kinetic SDE to obtain a more accurate update at no additional cost.

The kinetic SDE is linear in the state (V_t, L_t) except for the gradient term $\nabla f(L_t)$. If we freeze the gradient at its value at the start of the interval, replacing $\nabla f(L_t)$ by the constant $\nabla f(\vartheta_k)$ for all $t \in [kh, (k+1)h]$, the SDE becomes linear and admits an explicit closed-form

solution. The KLMC update matches the exact endpoint of this linearised SDE; the only approximation introduced is the gradient-freezing itself.

Within each interval, the velocity equation becomes

$$dV_t = (-\gamma V_t - \nabla f(\vartheta_k)) dt + \sqrt{2\gamma} dW_t,$$

which is an Ornstein–Uhlenbeck process with constant external forcing. Its solution is obtained via an integrating factor:

$$V_{kh+t} = e^{-\gamma t} V_{kh} - \left(\int_0^t e^{-\gamma(t-s)} ds \right) \nabla f(\vartheta_k) + \sqrt{2\gamma} \int_0^t e^{-\gamma(t-s)} dW_{kh+s}.$$

The position is recovered by integrating the velocity, $L_{kh+t} = L_{kh} + \int_0^t V_{kh+s} ds$. Integrating this expression yields a closed form in terms of V_{kh} , L_{kh} , $\nabla f(\vartheta_k)$, and an integrated Brownian term. To clarify the update rule, we isolate the time-dependent coefficients and define:

$$\psi_0(t) = e^{-\gamma t}, \quad \psi_1(t) = \int_0^t \psi_0(s) ds = \frac{1 - e^{-\gamma t}}{\gamma}, \quad \psi_2(t) = \int_0^t \psi_1(s) ds = \frac{\gamma t - 1 + e^{-\gamma t}}{\gamma^2}. \quad (19)$$

Each coefficient has a physical interpretation:

- $\psi_0(t)$ defines the exponential decay of the initial velocity under friction over an interval of length t .
- $\psi_1(t)$ determines the spatial displacement driven by a unit initial velocity vector over the interval.
- $\psi_2(t)$ maps the spatial displacement produced by a unit constant gradient force acting throughout the interval.

The KLMC algorithm generates a sequence $(v_k, \vartheta_k)_{k \in \mathbb{N}}$ in $\mathbb{R}^d \times \mathbb{R}^d$ via the recursion:

$$\begin{pmatrix} v_{k+1} \\ \vartheta_{k+1} \end{pmatrix} = \begin{pmatrix} \psi_0(h) v_k - \psi_1(h) \nabla f(\vartheta_k) \\ \vartheta_k + \psi_1(h) v_k - \psi_2(h) \nabla f(\vartheta_k) \end{pmatrix} + \begin{pmatrix} \xi_{k+1} \\ \xi'_{k+1} \end{pmatrix}, \quad \begin{pmatrix} \xi_{k+1} \\ \xi'_{k+1} \end{pmatrix} \sim \mathcal{N}(0, \Xi(h, \gamma)). \quad (20)$$

Following this formulation, the new velocity is the prior velocity damped by $\psi_0(h)$ minus the integrated gradient force $\psi_1(h) \nabla f(\vartheta_k)$. The new position is the prior position plus the displacement due to the prior velocity, $\psi_1(h) v_k$, minus the displacement produced by the gradient force, $\psi_2(h) \nabla f(\vartheta_k)$. After running the recursion, the position iterates $\{\vartheta_k\}$ are retained as samples from the target, and the auxiliary velocities $\{v_k\}$ are discarded.

The Gaussian pair (ξ_{k+1}, ξ'_{k+1}) does not consist of independent variables. Because the underlying Brownian path contributes to both the velocity and the position at time $(k+1)h$, the components are correlated. The distribution for this pair is a centred bivariate Gaussian whose covariance matrix $\Xi(h, \gamma)$ is given in Dalalyan and Riou-Durand [1, Section 3]. In our Python implementation, this block covariance matrix is precomputed once at initialisation and applied via its Cholesky factor to sample the joint vector.

Each iteration of KLMC requires one evaluation of the gradient $\nabla f(\vartheta_k)$, the simulation of a single correlated $2d$ -dimensional Gaussian draw, and standard vector operations. The per-iteration cost is comparable to that of LMC; the computational savings stem from the reduced iteration complexity.

4.4 Convergence Theory

To compare KLMC with LMC, we require a quantitative bound on the error after k iterations. Here, we state the mixing rate of the continuous-time kinetic diffusion and then incorporate the discretisation error introduced by the gradient-freezing. Combining these components yields an iteration count K_{KLMC} needed to reach the chosen error ε , which can be evaluated against the count K_{LMC} from Section 1. Both counts exhibit different dependences on the dimension d , the precision ε , and the condition number $\kappa = M/m$.

The error is measured in the Wasserstein-2 distance $\mathcal{W}_2$, a metric on probability distributions. While the earlier LMC bound in Section 1 used total variation, we switch to Wasserstein-2 ($\mathcal{W}_2$) here, the natural metric for the kinetic bounds; under strong log-concavity, convergence in $\mathcal{W}_2$ and in total variation are closely related.

Assumption 4.1. We assume f is twice differentiable with

$$mI_d \preceq \nabla^2 f \preceq MI_d.$$

Mixing of the continuous diffusion

Consider two copies of the kinetic SDE (17) driven by the same Brownian motion but started from different initial distributions μ and μ' . As both processes evolve, they converge toward the common invariant law p_* . The rate of this contraction defines the mixing rate of the diffusion.

Theorem 4.1 (1, Theorem 1). *For every $\gamma > 0$ and $t \geq 0$, there exist constants $\alpha \leq \sqrt{2}/\gamma$ and $\beta \geq (m \wedge (\gamma^2 - M))/\gamma$ such that for any μ, μ' on $\mathbb{R}^{2d}$,*

$$\mathcal{W}_2(\mu P_t^L, \mu' P_t^L) \leq \alpha e^{-\beta t} \mathcal{W}_2(\mu, \mu').$$

The contraction rate β is positive when $\gamma^2 > M$, indicating that the friction must be large enough to damp the slowest mode of the system. The lower bound on β reflects a physical trade-off: small γ leaves the system underdamped, causing the process to oscillate rather than mix, while large γ over-damps the velocity, causing the dynamics to slow down. Optimising the bound over γ yields $\gamma^* = \sqrt{m + M}$ and $\beta^* = m/\sqrt{m + M}$, where these two effects balance. This optimal friction is used in the subsequent complexity analysis, and its sensitivity is examined empirically in Section 5.4.

Error of the discretised algorithm

The convergence analysis of the algorithm must account for the bias introduced by the gradient-freezing approximation. The following result decomposes the total error into an initialization term and a discretisation term.

Theorem 4.2 (1, Theorem 2). *Let ν_k be the law of the k -th position iterate ϑ_k , with $(v_0, \vartheta_0) \sim \mathcal{N}(0_d, I_d) \otimes \nu_0$. For $\gamma \geq \sqrt{m + M}$ and $h \leq m/(4\gamma M)$,*

$$\mathcal{W}_2(\nu_k, \pi) \leq \sqrt{2} \left(1 - \frac{0.75mh}{\gamma}\right)^k \mathcal{W}_2(\nu_0, \pi) + \frac{Mh\sqrt{2d}}{m}.$$

The first term decays because the contraction factor $1 - 0.75mh/\gamma$ is strictly less than one, shrinking the initialization error exponentially. The second term is constant in k and represents an asymptotic bias floor: running the algorithm forever cannot reduce the error below $Mh\sqrt{2d}/m$. This floor scales with the step size and with $\sqrt{d}$. Reducing h to lower this floor slows the geometric decay of the first term, requiring an explicit balance between the two terms.

Iteration count and comparison

To bring the total error below ε , we bound each of the two terms by $\varepsilon/2$. Setting the second term equal to $\varepsilon/2$ determines the step size:

$$\frac{Mh\sqrt{2d}}{m} \leq \frac{\varepsilon}{2} \iff h \leq \frac{m\varepsilon}{2M\sqrt{2d}}.$$

With h fixed, solving for the number of iterations k required for the geometric term to fall below $\varepsilon/2$ yields an expression logarithmic in $\mathcal{W}_2(\nu_0, \pi)/\varepsilon$. Substituting h and $\gamma = \sqrt{m+M}$ yields the complexity bound:

$$K_{\text{KLMC}} \gtrsim \kappa^{3/2} \frac{\sqrt{d}}{\varepsilon} \log\left(\frac{\mathcal{W}_2(\nu_0, \pi)}{\varepsilon}\right). \quad (21)$$

The corresponding LMC iteration count for the strongly log-concave setting, established by Durmus and Moulines [4] and Dalalyan [3], is

$$K_{\text{LMC}} \gtrsim \kappa \frac{d}{\varepsilon^2} \log\left(\frac{\mathcal{W}_2(\nu_0, \pi)}{\varepsilon}\right). \quad (22)$$

These bounds are summarized in Table 1.

Algorithm	Iterations	in d	in ε
LMC	$\kappa d/\varepsilon^2$	d	$1/\varepsilon^2$
KLMC	$\kappa^{3/2}\sqrt{d}/\varepsilon$	$\sqrt{d}$	$1/\varepsilon$

Table 1 Iteration counts to reach ε error in the respective metric, under $mI_d \preceq \nabla^2 f \preceq MI_d$, with $\kappa = M/m$. Logarithmic factors are suppressed.

So KLMC reduces the dimension dependence from d to $\sqrt{d}$ and the precision dependence from $1/\varepsilon^2$ to $1/\varepsilon$. This matters most when the dimension is large or when high precision is required. Conversely, KLMC exhibits a worse dependence on the condition number, scaling as $\kappa^{3/2}$ compared to κ for LMC. The theoretical crossover occurs at $\kappa \asymp d/\varepsilon^2$. When this value is small, KLMC’s dimension and precision advantages dominate; on poorly conditioned targets where κ is large, LMC retains a theoretical advantage. Since these are upper bounds, empirical validation is presented in Section 5.4.

4.5 Validation of Theory

The three predictions of Theorems 4.1 and 4.2 on optimal friction $\gamma^* = \sqrt{m+M}$, dimension scaling $\sqrt{d}$, and precision scaling $1/\varepsilon$ are tested empirically in Section 5.4.

4.6 Beyond the Strongly Log-Concave Setting

The analysis above relies on f being globally m -strongly convex, which excludes targets with multiple modes or non-convex regions. Extending non-asymptotic guarantees to non-convex potentials remains an active area of research. We note two standard approaches. Dalalyan [3] (Section 4.3) demonstrates how to recover global strong convexity by adding a quadratic penalty outside a compact ball of radius R centered at the mode; sampling from the modified target remains close in total variation to the original distribution, provided the target mass outside the ball is negligible.

4.7 Discussion

The complexity bounds in Section 4.4 make the trade-off between LMC and KLMC fairly clean. LMC needs on the order of $\kappa d/\varepsilon^2$ iterations and KLMC needs $\kappa^{3/2}\sqrt{d}/\varepsilon$, so the comparison hinges on which factor dominates. KLMC wins on dimension and precision and pays the cost on conditioning: its $\kappa^{3/2}$ is worse than LMC’s κ , and the practical crossover sits roughly at $\kappa \approx d/\varepsilon^2$. On well-conditioned, moderate-dimensional problems, which is most of what we test in Sections 5.1 and 5.2, LMC is the more efficient choice. Push the same problems to tighter accuracy or to higher dimensions and the precision experiment of Section 5.4 shows what happens: the iteration counts cross over.

The friction parameter is the most delicate aspect of KLMC in practice. Theorem 4.1 gives the optimum as $\gamma^* = \sqrt{m + M}$, but that needs m and M , which on a real posterior we essentially never know. The friction sweep is reassuring here. The iteration count is reasonably flat over a wide range of γ below the optimum, and the sharp degradation only happens when γ is mistuned upward. So if a practitioner can put a rough upper bound on the Hessian spectrum, which for a strongly log-concave model is usually doable from the prior and data scale, they can pick γ at the small end of the plausible range and not lose much. That is not a substitute for knowing the constants, but it is a reason the algorithm is usable.

One direction we haven’t pursued is the second-order KLMC of Dalalyan and Riou-Durand [1, Section 4]. This incorporates the Hessian into the discretisation and improves the iteration count from $\sqrt{d}/\varepsilon$ to $\sqrt{d}/\varepsilon$ when the Hessian is itself Lipschitz. The per-iteration cost goes up because you need Hessian-vector products, but for moderate d this is often worthwhile. This would require computing Hessian-vector products and tracking a third Lipschitz constant, which was outside the scope of this initial implementation.

5 Results

This section compares ULA, MALA, and KLMC on three targets: a strongly correlated Gaussian distribution, Bayesian linear regression, and Bayesian logistic regression. The aim is empirical: to show how accuracy, mixing, and computational cost change across the tested step-size grids. These sweeps should be read as tuning curves rather than as comparisons at one arbitrary value of h .

The implementation team’s empirical comparisons (linear and logistic regression) are joined by experiments testing the theoretical predictions of KLMC from Section 4.

Diagnostics and notation

Section 3.4 defines autocorrelation, integrated autocorrelation time, and effective sample size. Here we only fix the notation used in the results figures. Let μ and Σ denote the exact posterior mean and covariance when they are available. In the logistic-regression experiment, where exact posterior moments are unavailable, these are replaced by long-chain reference estimates. For a post-burn-in chain with sample mean $\hat{\mu}$ and sample covariance $\hat{\Sigma}$, we report

$$e_\mu = \|\hat{\mu} - \mu\|_2, \quad e_\Sigma = \|\hat{\Sigma} - \Sigma\|_F.$$

The mean error e_μ measures accuracy in the posterior centre, while the covariance error e_Σ measures accuracy in the posterior spread and correlation structure. We use the absolute Frobenius covariance error rather than the relative error $\|\hat{\Sigma} - \Sigma\|_F/\|\Sigma\|_F$, because the main comparisons

are made within a fixed target and the absolute covariance distortion most directly displays the finite-step bias of the unadjusted methods.

Mixing is measured using the minimum coordinate effective sample size,

$$\text{ESS}_{\min} = \min_{1 \leq j \leq d} \text{ESS}_j,$$

which gives a conservative summary of the worst-mixing coordinate. Computational efficiency is measured by the cost-normalised quantity

$$C = N_{\nabla} + N_{\log \pi}, \quad \text{ESS}_{\text{cost}} = \frac{\text{ESS}_{\min}}{C},$$

where N_{∇} counts gradient evaluations and $N_{\log \pi}$ counts log-density evaluations. Thus e_{μ} and e_{Σ} measure accuracy, $\text{ESS}_{\min}$ measures raw mixing, and ESS_{cost} measures effective mixing per unit computational work. This cost model is not a wall-clock runtime model, but it prevents the Metropolis correction in MALA from being treated as free.

5.1 A first benchmark: a strongly correlated Gaussian

The first test target is the bivariate Gaussian with mean zero and covariance

$$\Sigma = \begin{pmatrix} 1 & 0.9 \\ 0.9 & 1 \end{pmatrix},$$

for which

$$\log \pi(x) = -\frac{1}{2}x^{\top} \Sigma^{-1}x, \quad \nabla \log \pi(x) = -\Sigma^{-1}x.$$

This is a deliberately demanding target: the strong correlation produces a narrow, tilted density, so accurately estimating the covariance is more informative than merely estimating the centre. Since the exact mean and covariance are known, accuracy can be scored directly rather than against a numerical reference.

As an initial sanity check, we run MALA for $N = 20\,000$ iterations from the origin with step size $h = 0.25$, discarding the first 2 000 samples as burn-in. The acceptance rate is 0.393, the post-burn-in sample mean is $(-0.0035, -0.0027)$, and the sample covariance is

$$\begin{pmatrix} 0.988 & 0.887 \\ 0.887 & 0.982 \end{pmatrix},$$

which is close to the target covariance. This confirms that the sampler and diagnostics behave as expected before comparing methods.

Figure 1 shows no visible drift in the trace and an autocorrelation function that decays within a modest number of lags. We then run ULA, MALA, and KLMC from the same starting point across a shared step-size grid. Each run uses $N = 12\,000$ iterations with 2 000 discarded as burn-in. For each value of h , we record mean error, covariance error, minimum-coordinate ESS, cost-normalised minimum-coordinate ESS, and the MALA acceptance rate.

Figure 2 reports the full step-size sweep for ULA, MALA, and KLMC. The sweep should be read as an accuracy–mixing–cost trade-off rather than as a ranking by any single panel. In particular, increasing h can improve movement through the state space and hence increase ESS, but for unadjusted methods this may also increase the bias in the sampled distribution. The key question is therefore whether gains in ESS occur while the mean and covariance errors remain controlled.

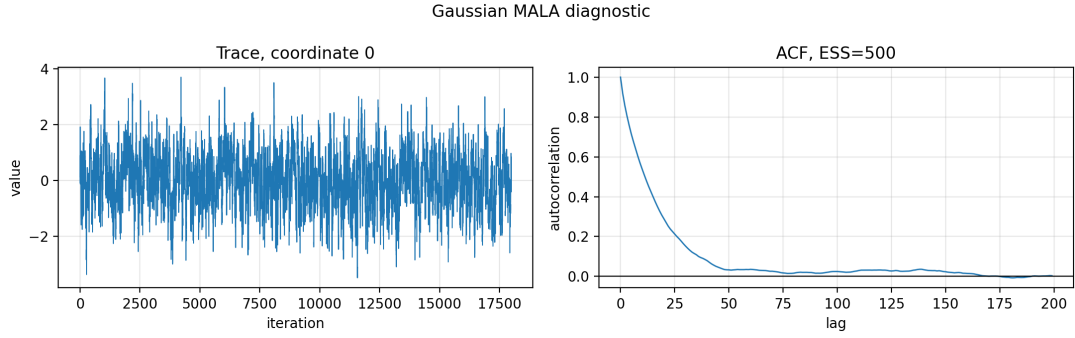


Figure 1 Diagnostics for a single MALA run on the correlated Gaussian target. The left panel shows the trace of the first coordinate, and the right panel shows its normalised autocorrelation function.

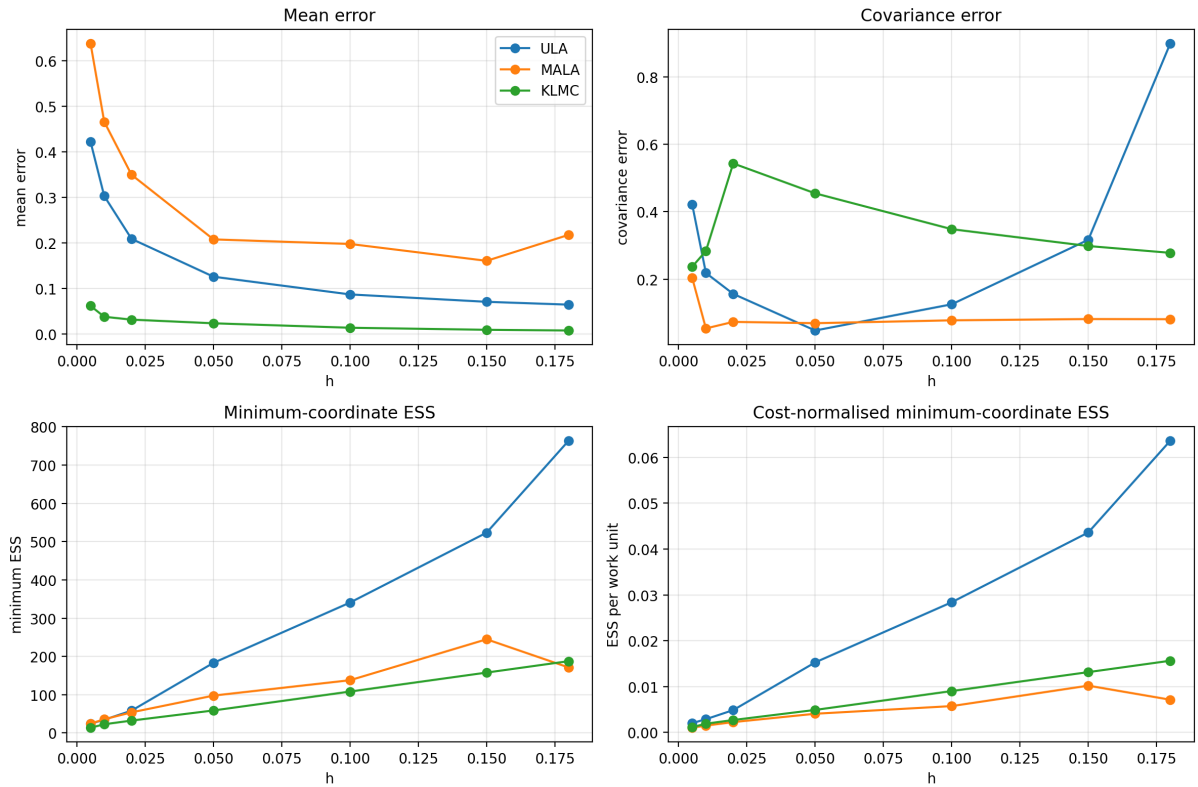


Figure 2 Step-size sweep for ULA, MALA, and KLMC on the strongly correlated Gaussian target. The panels show posterior mean error, covariance error, minimum-coordinate ESS, and cost-normalised minimum-coordinate ESS. The first two panels measure accuracy, the third measures raw mixing, and the fourth measures computational efficiency under the cost model defined above.

The mean-error panel is reassuring but not the most discriminating diagnostic here. Since the true mean is zero, all three samplers estimate the posterior centre reasonably well once they explore the main mass of the target. KLMC has the smallest mean error across the grid, and ULA's mean error also decreases as h grows. However, the main challenge in this benchmark is capturing the tilted covariance structure, so the covariance error is the more informative accuracy diagnostic.

The covariance panel reveals the key distinction between adjusted and unadjusted methods. ULA's covariance error initially decreases as the step size increases, but then grows sharply for larger h . This is the finite-step bias of the Euler discretisation: larger steps improve movement through the state space, but the invariant distribution of the discretised chain is no longer exactly π . MALA behaves differently. Its covariance error remains comparatively small and stable over the useful part of the grid, because the Metropolis correction removes the finite-step invariant-measure bias of the Langevin proposal, although finite-run Monte Carlo error remains. KLMC benefits from momentum and has strong mean accuracy, but as an unadjusted method its covariance accuracy remains step-size dependent.

Comparing raw minimum-coordinate ESS, ULA appears strongest at large step sizes: it never rejects, so increasing h produces larger moves and faster decorrelation. MALA's ESS is lower and eventually begins to turn over, as larger proposals reduce the acceptance probability and rejected proposals repeat the current state. KLMC's raw ESS also increases with h , but it remains below ULA throughout the grid and is below MALA except at the largest step size. Thus, on this Gaussian benchmark, the momentum variable does not translate into superior raw mixing.

The cost-normalised ESS panel gives the corresponding computational efficiency comparison. Since MALA requires additional log-density and proposal-ratio work for the accept-reject step, its ESS per work unit is lower than its raw ESS alone would suggest. ULA remains the most efficient method by this diagnostic at large h , while KLMC becomes competitive with MALA. However, this ranking cannot be read in isolation: ULA's large ESS per work unit at the largest step sizes coincides with a large covariance error. It is therefore efficiently sampling from a biased finite-step approximation rather than accurately sampling from the target.

This benchmark establishes the main empirical lesson for the later experiments. Mean and covariance errors assess accuracy, minimum-coordinate ESS measures raw mixing, and cost-normalised ESS measures computational efficiency. ULA is cheap and can mix rapidly, but it is reliable only when the step size is small enough that discretisation bias is controlled. MALA pays for the Metropolis correction through extra work and possible rejections, but this correction gives more stable covariance accuracy. KLMC can improve exploration through momentum and is competitive in cost-normalised ESS, but in this unadjusted implementation it remains sensitive to the choice of step size.

5.2 Bayesian linear regression

The Bayesian linear regression experiment repeats the comparison in a slightly higher dimensional setting where the target remains exactly tractable. We simulate a regression problem with $d = 5$ coefficients and a Gaussian prior, so the posterior mean and covariance are available in closed form from Section 2.8. Thus, we are able to compute exact posterior moment errors, rather than errors against a numerical reference.

Figure 3 broadly reinforces the trade-off observed in the correlated Gaussian benchmark. The mean errors are small across the useful range of step sizes, and ULA achieves the lowest mean error at the larger values of h . However, the covariance panel shows the more important accuracy distinction. ULA's covariance error increases sharply at the largest step sizes, showing the finite-step bias of the unadjusted Euler discretisation. MALA keeps the covariance error

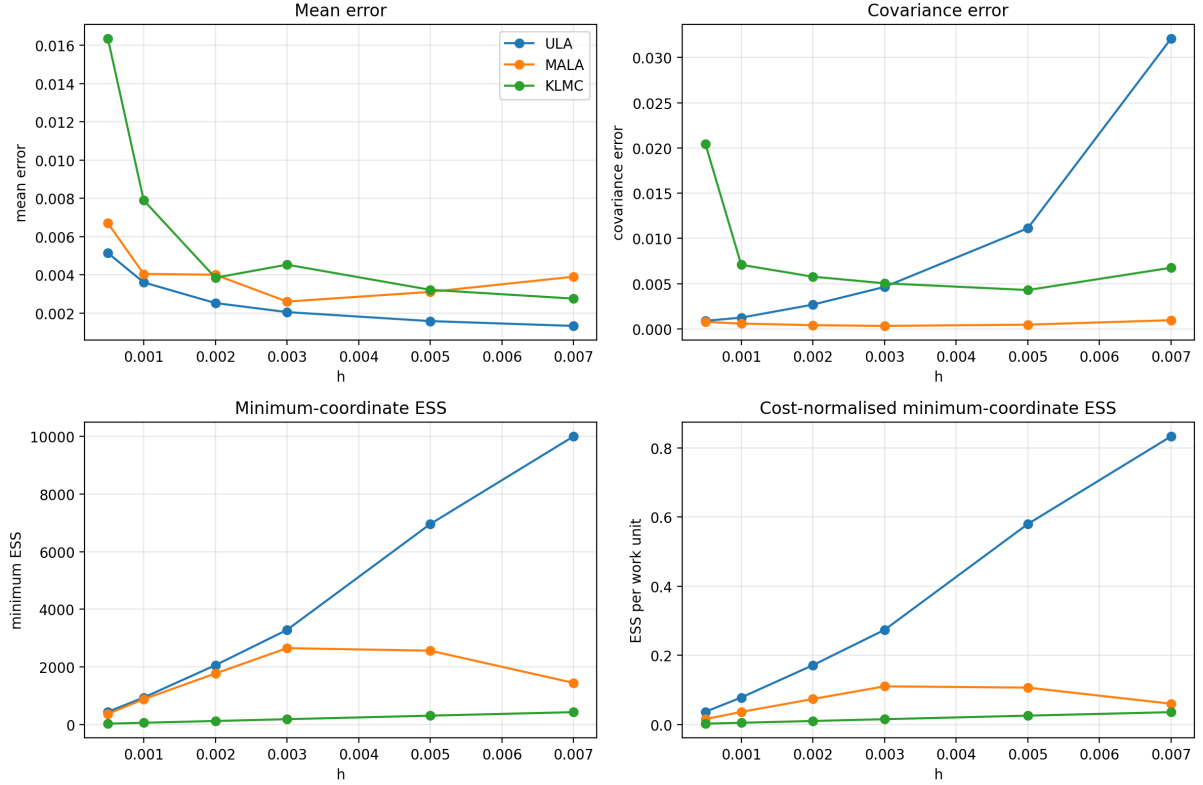


Figure 3 Step-size sweep for ULA, MALA, and KLMC on the Bayesian linear regression posterior.

close to zero throughout the grid, so it gives the most reliable estimate of posterior uncertainty in this Gaussian regression target. KLMC improves substantially as h increases from very small values, but with the fixed friction parameter used here it does not edge out MALA on covariance accuracy.

The ESS panels separate mixing from accuracy. ULA achieves the largest minimum-coordinate ESS and the largest ESS per work unit as h increases, but this efficiency gain coincides with worsening covariance error at the largest step sizes. MALA has a more moderate ESS profile: raw and cost-normalised ESS improve at first and then fall at larger h . Thus the largest MALA step sizes are rejection-heavy, so the Metropolis correction preserves covariance accuracy but reduces effective movement through repeated rejections. KLMC’s ESS increases steadily with h under the fixed friction parameter used here, but it loses out to the other methods on this benchmark. Overall, the linear regression experiment confirms that the sampler with the highest ESS is not necessarily the most accurate one: ULA is most efficient by the ESS diagnostics, while MALA is most stable for covariance estimation.

5.3 Bayesian logistic regression

For Bayesian logistic regression, exact posterior moments are not available in closed form. We therefore use a long small-step MALA chain to obtain reference estimates μ_{ref} and Σ_{ref} of the posterior mean and covariance. MALA is used for this reference because the Metropolis correction removes the finite-step invariant-measure bias of the Langevin proposal, although finite-run Monte Carlo error remains. The mean and covariance errors reported below are therefore errors relative to this numerical reference, not exact posterior errors.

The shorter ULA, MALA, and KLMC runs are scored against $(\mu_{\text{ref}}, \Sigma_{\text{ref}})$. Figure 4 shows

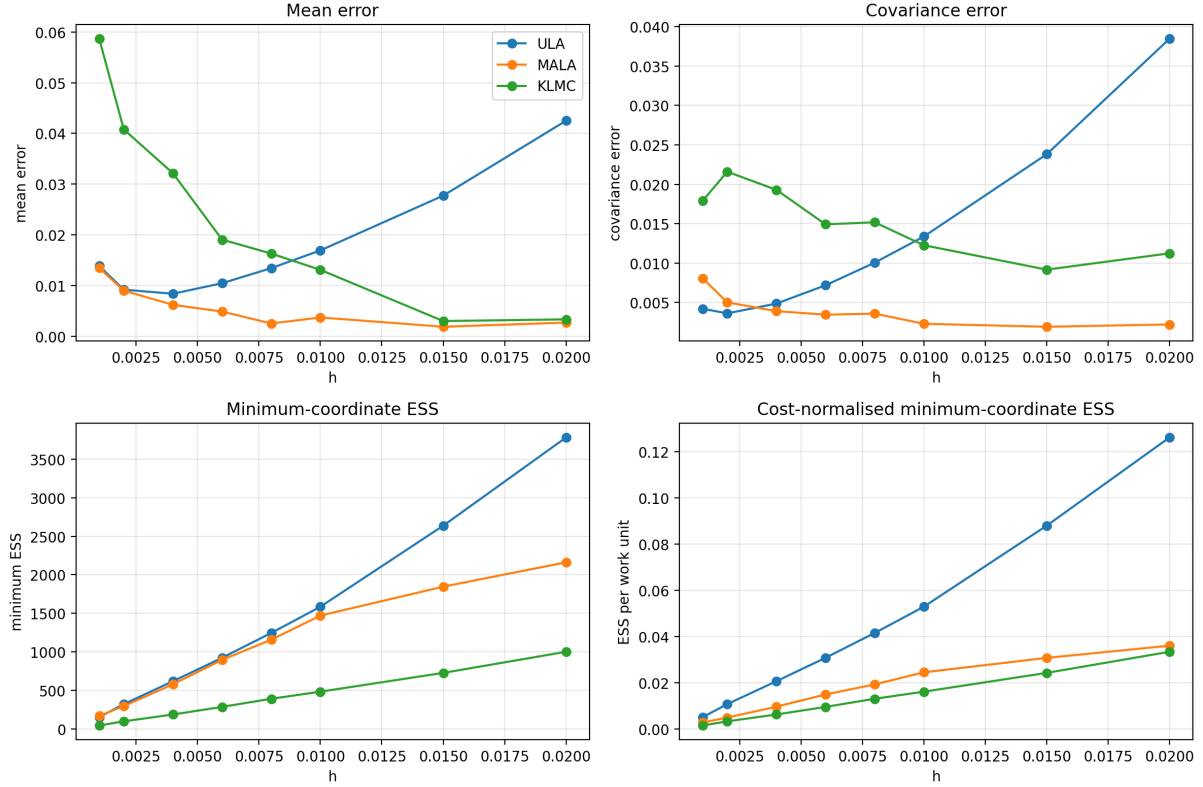


Figure 4 Step-size sweep for ULA, MALA, and KLMC on the Bayesian logistic regression posterior.

that the same accuracy–mixing–cost trade-off persists in the non-Gaussian model. ULA achieves the largest minimum-coordinate ESS and the largest ESS per work unit as h increases, but the accuracy panels show the cost of this apparent efficiency. Both its mean error and covariance error grow noticeably at the largest step sizes, indicating that the unadjusted discretisation is moving quickly but no longer matching the reference posterior moments well.

MALA gives the most stable accuracy in this experiment. Its mean error remains small across the larger step sizes, and its covariance error stays close to zero throughout the grid. The acceptance rates decrease from 0.994 at $h = 0.001$ to 0.561 at $h = 0.020$, so the displayed grid has not entered a severely rejection-dominated regime. This explains why both raw and cost-normalised ESS continue to increase with h . MALA is therefore less efficient than ULA by the ESS-per-work diagnostic, but it gives a much more reliable approximation to the reference posterior moments.

KLMC behaves differently from both ULA and MALA. Its mean error is large for very small step sizes but decreases sharply as h increases, eventually becoming comparable with MALA in mean accuracy. Its covariance error also improves over most of the grid and remains well below ULA’s error at the largest step sizes, but it does not match MALA’s covariance accuracy. Its raw ESS remains below both ULA and MALA, although its cost-normalised ESS approaches MALA’s at the largest tested step sizes. Under the fixed friction parameter used here, KLMC is therefore competitive for sufficiently large h , but its accuracy is more sensitive to tuning than MALA’s.

Overall, the logistic regression benchmark supports the same conclusion as the Gaussian experiments, now in a non-Gaussian posterior where the truth must be approximated by a reference chain. ULA is attractive by the ESS diagnostics but becomes unreliable at larger step sizes. MALA gives the strongest accuracy against the reference posterior moments, while KLMC offers improving cost-aware exploration as h increases but does not dominate MALA on

covariance accuracy. The sampler with the largest ESS is therefore not necessarily the best approximation to the posterior.

Across all three experiments, the same pattern appears. ULA is cheap and can produce high ESS at large step sizes, but the accuracy diagnostics reveal the finite-step bias of the unadjusted discretisation. MALA has lower cost-normalised ESS when the Metropolis correction and rejection behaviour are accounted for, but it gives more reliable posterior moment accuracy. KLMC can improve exploration through momentum, but in the unadjusted implementation it remains sensitive to the chosen step size and friction parameter. These results show why the samplers cannot be ranked by ESS alone: accuracy, raw mixing, and computational efficiency must be considered together.

5.4 KLMC: Friction, Dimension, and Precision Scaling

The convergence theory makes three verifiable predictions: an optimal friction parameter $\gamma^* = \sqrt{m + M}$, an iteration count scaling as $\sqrt{d}$ in the dimension, and a scaling as $1/\varepsilon$ in the precision. The preceding subsections on the three benchmark targets demonstrate that KLMC achieves a higher effective sample size per gradient evaluation across a range of step sizes. This subsection outlines three targeted numerical experiments designed to test the theoretical scaling predictions directly. This subsection outlines three targeted numerical experiments designed to test the theoretical scaling predictions directly.

All experiments use the implementation details from Section 3. Each experiment uses a problem-specific diagnostic, described below.

Friction parameter sweep

The first experiment tests the prediction of Theorem 4.1 that the mixing rate of the continuous-time process is maximised at $\gamma^* = \sqrt{m + M}$. We hold the step size h fixed and sweep γ across a logarithmic grid, recording for each value the smallest iteration count K at which the running empirical covariance falls within Frobenius distance ε of the target, where ε is set to a fixed fraction of $\|\Sigma\|_F$ so the threshold scales with the problem.

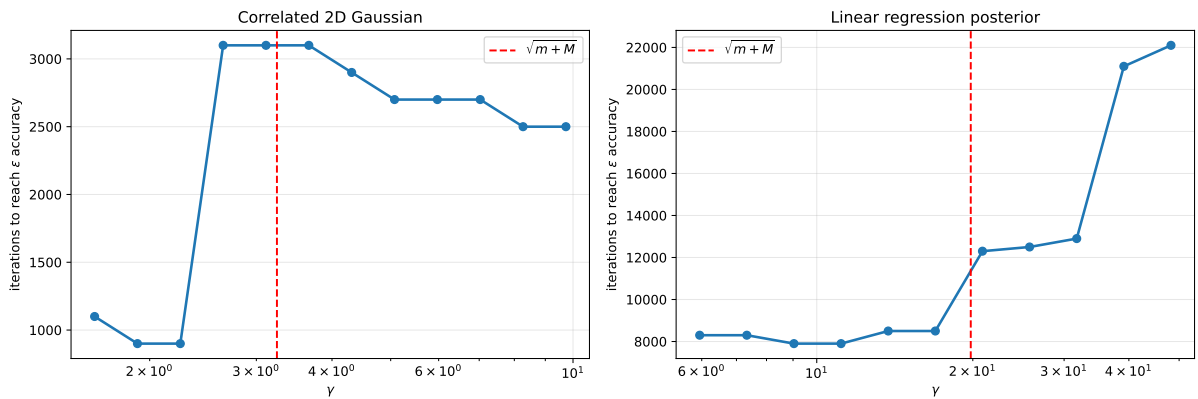


Figure 5 Friction parameter sweep on the correlated Gaussian (left) and the Bayesian linear regression posterior (right). The vertical axis reports the iteration count K required for the running empirical covariance to first fall within ε of the target. The red dashed line marks the theoretical optimum $\gamma^* = \sqrt{m + M}$ predicted by Theorem 4.1.

The friction sweep on the linear regression posterior gives a clear picture. With $m \approx 148$, $M \approx 245$, and $\sqrt{m + M} \approx 19.8$, the iteration counts sit flat at $K \approx 8000$ across $\gamma \in [6, 17]$ and

then start climbing fast. By $\gamma \approx 40$ the count has doubled, and at $\gamma \approx 60$ the chain doesn't converge in the iteration budget at all. The theoretical optimum sits just above the flat region, at the onset of the climb; the empirically cheapest runs lie on the plateau slightly below γ^* , so the theory locates the optimum correctly to within the resolution of the sweep.

The Gaussian target is less clean. With $m \approx 0.53$, $M \approx 10$, the iteration counts are actually lowest at the smallest γ tested ($K \approx 900$ at $\gamma \approx 1.9$), rise to a plateau near $K \approx 3000$ around $\sqrt{m+M}$, and recover slightly at larger γ . We should be cautious about the small- γ result: at this step size the algorithm is near the edge of the regime where Theorem 4.2 actually applies, so what looks like fast convergence might just be the chain settling into a biased stationary distribution. The two targets together suggest that $\gamma^* = \sqrt{m+M}$ is most reliable when the theoretical step-size bound is satisfied with room to spare, which the Gaussian target doesn't have. The practical takeaway is that mistuning γ downward (underdamping) is more forgiving than mistuning upward.

Scaling with the dimension

The second experiment tests the dimension scaling in Table 1: theory says LMC scales as d and KLMC as $\sqrt{d}$, so on a log-log plot of K against d we would expect slopes of 1 and 1/2 respectively.

To isolate the dimension we use isotropic Gaussian targets $\pi = \mathcal{N}(0, I_d)$ across $d \in \{3, 5, 8, 12, 20, 30, 50\}$, which keeps $m = M = 1$ and $\kappa = 1$. The diagnostic is the squared Wasserstein-2 distance, which has a closed form when both measures are Gaussian: for our target,

$$\mathcal{W}_2^2(\hat{\nu}_K, \pi) = \|\hat{\mu}_K\|_2^2 + \text{tr}(\hat{\Sigma}_K) + d - 2\text{tr}(\hat{\Sigma}_K^{1/2}),$$

where $\hat{\mu}_K, \hat{\Sigma}_K$ are the empirical mean and covariance after burn-in. This is the metric in which Theorem 4.2 is actually stated. We record the smallest K for which this distance first falls below 0.1, take the median over three seeds to cut down on noise, and use the step size $h = \min(0.15, \sqrt{0.8/d})$ so the LMC bias floor stays below the threshold.

The fitted slopes across $d \in [3, 30]$ come out as 1.04 for LMC and 1.23 for KLMC, against theoretical predictions of 1.0 and 0.5. The LMC slope is essentially the theoretical exponent, which is reassuring. KLMC is steeper than predicted, and at every d tested it takes slightly more iterations than LMC, which looks at first like it is contradicting the theory.

The gap is small in absolute terms though. At $d = 30$ KLMC needs about 37% more iterations than LMC, not orders of magnitude more, and there are a few reasons the asymptotic advantage doesn't show up here. The constants hidden in the $\gtrsim$ notation of (21) and (22) aren't symmetric: KLMC's bound carries a $\sqrt{2}$ in the contraction rate and a larger constant prefactor than LMC's, which pushes the actual crossover dimension above the simple $d \approx 1/\varepsilon^2$ that schematic comparisons might suggest. The Wasserstein estimator also has its own Monte Carlo variance that scales with d/K_{eff} , and this affects KLMC more because its momentum introduces extra autocorrelation. The qualitative story is still consistent with the theory though: both samplers grow polynomially in d , LMC's slope matches the theoretical prediction, and the regime where KLMC's $\sqrt{d}$ advantage becomes visible would need a bigger compute budget than we had.

Scaling with the precision

The third experiment fixes $d = 5$ and varies the target accuracy $\varepsilon \in \{0.6, 0.4, 0.25, 0.15, 0.08, 0.05\}$. Theorem 4.2 predicts that KLMC scales as $1/\varepsilon$ and LMC as $1/\varepsilon^2$, so the expected log-log slopes

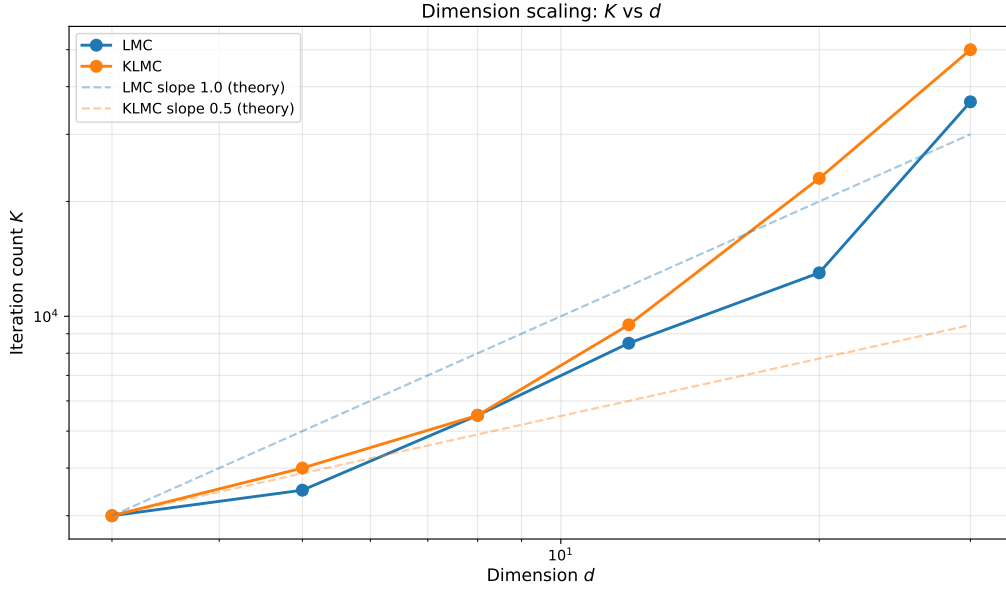


Figure 6 Median iteration count K against dimension d on log-log axes. Dashed lines mark the theoretical slopes 1 (LMC) and $1/2$ (KLMC), calibrated to the $d = 3$ data point. Neither sampler reaches the threshold at $d = 50$ within 80,000 iterations.

against $1/\varepsilon$ are 1 and 2. The step size is adapted as $h = \min(0.15, \sqrt{8\varepsilon^2/d})$ to keep the LMC bias under the threshold across the range.

The fitted slopes are 1.47 for LMC and 0.78 for KLMC, both shallower than the theoretical 2 and 1. The reason is that at small ε the Wasserstein estimator’s own variance becomes comparable to the threshold, so the running estimator dips below it by chance and the first-hit time underestimates the true mixing time. The ratio of the two slopes is $1.47/0.78 \approx 1.9$, which is close to the theoretical ratio of 2, so the relative scaling of the two algorithms is recovered even when the absolute exponents aren’t.

The clearer signal is in the crossover. At loose precisions ($\varepsilon \geq 0.25$) LMC needs fewer iterations, but the curves cross around $\varepsilon = 0.15$, and at the tightest precisions KLMC takes over. At $\varepsilon = 0.15$, KLMC reaches the threshold in about 74% of LMC’s iteration count, and at $\varepsilon = 0.08$ LMC needs roughly 47,000 iterations while KLMC gets there in around 30,000. This crossover is what the theory predicts: KLMC’s better dependence on ε takes over once ε is small enough for it to matter.

Summary

Taken together the three experiments support the predictions of Theorems 4.1 and 4.2, though to varying degrees. The friction sweep confirms the qualitative claim of Theorem 4.1: on the well-conditioned linear regression posterior the iteration counts are flat across the predicted plateau and rise sharply above $\sqrt{m} + \bar{M}$. The dimension experiment confirms LMC’s linear-in- d scaling almost exactly (empirical slope 1.04 against theory 1), but KLMC’s $\sqrt{d}$ advantage doesn’t show up at the dimensions our budget allowed, which is consistent with the theory’s asymptotic character. The precision experiment recovers the relative scaling (slope ratio 1.9 against theoretical 2) and shows the crossover the theory predicts: KLMC beats LMC once ε gets small enough. The exponents that emerge cleanly from the data match theory, and the ones that don’t can be explained without undermining the underlying claims.

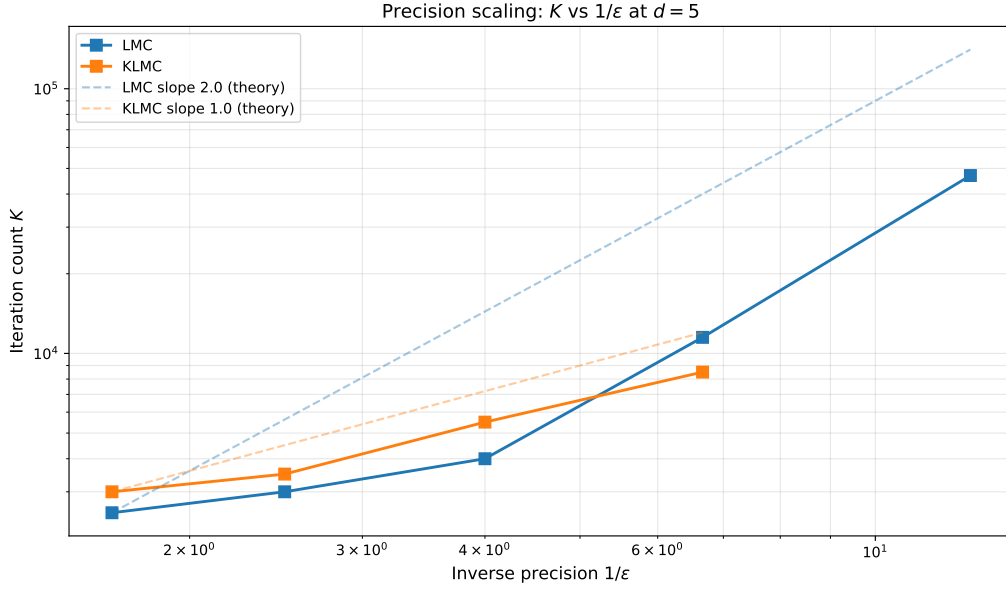


Figure 7 Median iteration count K against $1/\epsilon$ at $d = 5$, on log-log axes. Dashed lines mark theoretical slopes 2 (LMC) and 1 (KLMC), calibrated to the $\epsilon = 0.6$ point. The two curves cross near $\epsilon \approx 0.15$.

6 Discussion

The main interpretation is that the three algorithms should not be ranked by a single number. ULA is the simplest and cheapest discretisation, so it is attractive when a small amount of bias is acceptable or when the step size can be chosen conservatively. MALA uses the same local Langevin proposal but corrects it with a Metropolis–Hastings step, making it more reliable for exact targeting at the price of rejections and additional target/proposal computations. KLMC changes the underlying dynamics by adding momentum; this can improve exploration, but the unadjusted implementation is still sensitive to the integrator, the step size, and the friction parameter. There are several methodological caveats to consider when interpreting these numerical comparisons:

- **Minimum Coordinate-wise ESS:** The effective sample size reported is the minimum across all coordinates rather than a multivariate ESS. While this strictly conservative metric guarantees that the chain is mixing adequately in the worst direction, it likely under-reports the overall multivariate efficiency of the samplers. Furthermore, values near the post-burn-in chain length should be treated as diagnostic summaries rather than exact independent sample counts.
- **Cost Normalisation:** Efficiency is measured using the simple work count $C = N_{\nabla} + N_{\log \pi}$. This charges MALA for the log-density work needed by the accept–reject correction, but it still ignores proposal-density arithmetic, linear algebra constants, vectorisation, and implementation-level timing effects.
- **Empirical Ground Truth:** The logistic-regression errors are measured as distances to a long MALA reference chain rather than exact analytical posterior errors. Because this ground truth is empirical, finite-run Monte Carlo error in the reference chain is inherited by the reported mean and covariance errors. No multi-chain convergence or Monte Carlo standard error analysis is included.

- **Unadjusted KLMC:** KLMC is unadjusted in these experiments. Consequently, a high ESS can still correspond to the efficient exploration of a biased discretisation. KLMC’s apparently superior mixing performance in the plots cannot be taken at face value; it is only a genuine advantage if the step size is small enough to keep the unadjusted bias strictly bounded.

Method	Interpretation	Accuracy	Mixing and cost	Main weakness
ULA	Direct Euler discretisation of overdamped Langevin diffusion.	Biased for finite h ; exact only as $h \rightarrow 0$ in the limit.	One gradient per step; can have high raw ESS at large h .	Large steps can sample efficiently from the wrong law.
MALA	ULA proposal plus Metropolis–Hastings correction.	Targets π exactly asymptotically, after adequate mixing.	Extra target/proposal work per step; ESS depends on acceptance rate.	Poor tuning causes many rejections and slower exploration.
KLMC	Underdamped dynamics on position and velocity.	Integrator-dependent in the unadjusted implementation.	Momentum can improve cost-aware mixing.	Requires tuning h and γ ; theory depends on conditioning.

Table 2 Qualitative summary of the three Langevin-based methods.

7 Conclusion

This project studies Langevin-based MCMC as a practical approximation to an ideal continuous-time diffusion. The continuous-time overdamped Langevin process has the target distribution as its invariant law, but it cannot usually be simulated exactly. Discretisation makes simulation possible and leads to ULA, but it also introduces finite-step bias. MALA responds by adding a Metropolis correction to the same proposal, removing the asymptotic bias at the cost of rejections and additional target/proposal work. KLMC responds differently by augmenting the dynamics with momentum, which can improve exploration but introduces a friction parameter and a more complex discretisation. The experiments confirm that the best sampler is not simply the one with the highest ESS. Efficient sampling requires accurate estimation after burn-in at reasonable computational cost. ULA, MALA, and KLMC each navigate this trade-off differently, so the right comparison must report accuracy, burn-in and mixing diagnostics, acceptance behaviour where relevant, and cost per effective sample.

References

- [1] Arnak S. Dalalyan and Lionel Riou-Durand. On sampling from a log-concave density using kinetic Langevin diffusions. *Bernoulli*, 26(3):1956–1988, 2020. doi: 10.3150/19-BEJ1178.
- [2] Gareth O. Roberts and Richard L. Tweedie. Exponential convergence of Langevin distributions and their discrete approximations. *Bernoulli*, 2(4):341–363, 1996.
- [3] Arnak S. Dalalyan. Theoretical guarantees for approximate sampling from smooth and log-concave densities. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 79(3):651–676, 2017. doi: 10.1111/rssb.12183.

- [4] Alain Durmus and Éric Moulines. Nonasymptotic convergence analysis for the unadjusted Langevin algorithm. *The Annals of Applied Probability*, 27(3):1551–1587, 2017. doi: 10.1214/16-AAP1238.
- [5] sampling-using-SDEs. <https://github.com/maxcyn/sampling-using-SDEs>, 2026. GitHub repository.
- [6] Andrew Duncan. M4A44: Computational stochastic processes. Lecture notes, Imperial College London, 2016. URL https://urbain.vaes.uk/static/teaching/lectures/notes_2016.pdf.
- [7] Xiang Cheng, Niladri S. Chatterji, Peter L. Bartlett, and Michael I. Jordan. Underdamped Langevin MCMC: A non-asymptotic analysis. In *Proceedings of the 31st Conference on Learning Theory*, volume 75 of *Proceedings of Machine Learning Research*, pages 300–323. PMLR, 2018. URL <https://proceedings.mlr.press/v75/cheng18a.html>.
- [8] Edward Nelson. *Dynamical Theories of Brownian Motion*. Princeton University Press, 1967. doi: 10.1515/9780691219615.